

Compiling Metal

A Pedagogical Textbook on CAM Languages, Intermediate Representations, Geometry,
Optimization, and Machine-Checkable Assurance

Prepared for the Dropcut and Makera Z1 project

August 2026

Contents

Preface	i
The recurring job	ii
How to read the book	ii
1 What Are We Compiling?	1
1.1 Begin with the failure, not the file	1
1.1.1 The first design principle	2
1.2 A compiler for a physical process	3
1.2.1 The semantic waist	3
1.3 The physical vocabulary: solids, frames, paths, and stock	5
1.3.1 Solids	5
1.3.2 Frames	6
1.3.3 Paths and trajectories	7
1.3.4 Material-removal state	7
1.4 Manufacturing intent denotes a set, not one path	8
1.4.1 Planner as witness constructor	8
1.4.2 Permitted nondeterminism	9
1.5 Four complementary semantics	9
1.5.1 Denotational semantics: what does it mean?	10
1.5.2 Operational semantics: how does it run?	10
1.5.3 Axiomatic semantics: what must be true before and after?	10
1.5.4 Trace and temporal semantics: what must always or eventually happen?	11
1.5.5 Why all four are needed	11
1.6 State: what the next command depends on	12
1.6.1 Modal state	12
1.6.2 Process state and stock state	12
1.7 Contracts, assertions, invariants, and assumptions	13
1.7.1 Representation invariants versus physical guarantees	14
1.7.2 Weakest precondition intuition	14
1.8 Correctness as refinement	14
1.8.1 Exact preservation, refinement, and bounded refinement	15
1.8.2 Composition across passes	16
1.9 Error, uncertainty, and model reality	16
1.9.1 Numerical error	16
1.9.2 Model uncertainty	16
1.9.3 Process uncertainty	16
1.9.4 Inner and outer approximations	17
1.10 Worked derivation: the first semantic model of the pocket	17
1.10.1 Inputs	18
1.10.2 Source-level outcome predicate	18
1.10.3 Program-level trace predicate	19
1.10.4 End-to-end goal	19
1.11 Chapter summary	19
1.12 Exercises	19

1.12.1	Concept checks	19
1.12.2	Calculations	20
1.12.3	Modeling	20
1.12.4	Counterexample construction	20
1.12.5	Implementation project	20
2	Designing the Language, IR, and Compiler Passes	21
2.1	Why a pleasant API needs an austere core	21
2.1.1	The inert-AST rule	23
2.1.2	Isolation and termination	23
2.1.3	Scope combinators and asynchronous leakage	24
2.2	Units and frames as part of the type system	24
2.2.1	Constructors validate domains	24
2.2.2	Frame-parameterized geometry	25
2.3	Why several intermediate representations are necessary	26
2.3.1	Authoring AST	26
2.3.2	Elaborated Plan IR	26
2.3.3	Intent IR	26
2.3.4	Toolpath IR	28
2.3.5	Scheduled IR	28
2.3.6	Machine IR	28
2.3.7	Controller IR and job bundle	29
2.3.8	Legality at each level	29
2.4	Paths as a compositional algebra	30
2.4.1	Path image versus parameterization	31
2.4.2	Approximate equality is not equality	31
2.4.3	Reversal is semantic	32
2.5	A path is not an action: effects enter the language	32
2.5.1	Why ordinary function composition fails	32
2.5.2	Indexed effects and typestate	33
2.6	An SSA-style state token	34
2.6.1	Branches and measurements	34
2.7	Passes are relations, not functions with hopeful names	35
2.7.1	Common pass relations	35
2.7.2	Pass manifest	36
2.7.3	Translation validation	36
2.8	Provenance and content identity	37
2.8.1	Content addressing	37
2.8.2	Diagnostics as structured artifacts	38
2.9	The postprocessor is a compiler backend	38
2.9.1	Modal compression	39
2.9.2	Numeric formatting	39
2.9.3	Raw commands	39
2.10	Worked compilation of the recurring job	40
2.10.1	Authoring source	40
2.10.2	Elaborated Plan IR	40
2.10.3	Intent and toolpath IR	41
2.10.4	Scheduled and Machine IR	41
2.10.5	Controller IR and bytes	41
2.11	Applying the architecture to Dropcut Studio	41
2.12	Chapter summary	42
2.13	Exercises	43
2.13.1	Authoring language	43
2.13.2	Units and frames	43
2.13.3	IR design	43
2.13.4	Path algebra and effects	43

2.13.5	Compiler passes	43
2.13.6	Project	44
3	Planning Geometry, Motion, and Optimization	45
3.1	Planning begins with a feasible set	45
3.2	From tool shape and path to swept volume	46
3.2.1	Three distinct geometric obligations	47
3.2.2	Lines, arcs, and general curves	47
3.3	Configuration space makes linking intelligible	48
3.3.1	Why the current stock matters	48
3.3.2	Simple and advanced link policies	48
3.4	Stock representations and their proof power	49
3.4.1	Height fields	49
3.4.2	Dexels and triple dexels	49
3.4.3	Voxels and adaptive spatial trees	50
3.4.4	Boundary representations and exact queries	50
3.4.5	Choosing by claim	50
3.5	Sampling is evidence only when connected to a theorem	51
3.5.1	Lipschitz reasoning	51
3.5.2	Chord error for a circular arc	51
3.5.3	Adaptive refinement	52
3.6	Robust computational geometry	52
3.6.1	Exact topology, bounded geometry	53
3.6.2	Interval arithmetic	53
3.7	Error budgets are typed proof objects	54
3.7.1	Propagation through a pass	54
3.7.2	A sample budget	54
3.7.3	Budget-driven algorithms	56
3.8	Constructing a pocket path from first principles	56
3.8.1	Restating the intent	56
3.8.2	Step-over and coverage	56
3.8.3	Depth layers	57
3.8.4	Entry	57
3.8.5	Direction and climb milling	58
3.8.6	Linking contours	58
3.8.7	Pocket planner witness	58
3.9	Surface planning, distance fields, and the Eikonal equation	58
3.9.1	Cutter-location field	58
3.9.2	Scallop height intuition	59
3.9.3	Distance fields and Eikonal methods	59
3.9.4	Hybrid strategies	60
3.10	Scheduling as an operations-research problem	60
3.10.1	Precedence graph	60
3.10.2	Routing with precedence	61
3.10.3	Worked example: two legal schedules	61
3.10.4	Effects and commutativity	61
3.10.5	Feasibility checking	62
3.11	Feed scheduling and time-optimal parameterization	62
3.11.1	Curvature constraint	63
3.11.2	Controller versus compiler scheduling	63
3.12	Combining planning and checking	63
3.12.1	What simulation remains good for	64
3.13	Chapter summary	64
3.14	Exercises	64
3.14.1	Swept volumes and stock	64
3.14.2	Continuous bounds and robustness	65

3.14.3	Error budgets	65
3.14.4	Pocket planning	65
3.14.5	Surface planning	65
3.14.6	Operations research	65
3.14.7	Project	65
4	Building Trust: Invariants, Certificates, and Runtime Assurance	67
4.1	Start by naming the proposition	67
4.1.1	Separate proposition, method, and result	68
4.2	Hoare logic turns command requirements into derivations	69
4.2.1	Basic command contracts	69
4.2.2	Weakest preconditions	69
4.2.3	Worked derivation for the pocket epilogue	69
4.2.4	Loop invariants	70
4.3	Abstract interpretation checks all represented states	70
4.3.1	A small abstract machine domain	71
4.3.2	Transfer functions	71
4.3.3	Join and fixed point	71
4.3.4	Proof-carrying abstract states	72
4.4	Translation validation checks each actual pass result	72
4.4.1	Arc linearization checker	72
4.4.2	Path-reordering checker	73
4.4.3	Modal-compression checker	73
4.4.4	Why checkers can be smaller	73
4.5	The certificate graph	73
4.5.1	Schema	75
4.5.2	Assumptions	76
4.5.3	Evidence types	76
4.5.4	Dependency and invalidation	76
4.5.5	Completeness policies	77
4.6	Proof-carrying CAM and the trusted computing base	77
4.6.1	Proof object or recomputation?	78
4.6.2	Checker hardening	78
4.7	The exact bytes are the deployed artifact	78
4.7.1	Parse-back relation	79
4.7.2	Transport completeness	79
4.7.3	Controller storage identity	79
4.8	The controller is a concurrent state machine	80
4.8.1	Safety invariants	80
4.8.2	Liveness properties	80
4.8.3	Atomic admission	81
4.8.4	Ambiguous timeout	81
4.8.5	Stop-class commands	81
4.9	Runtime assurance binds proof to reality	82
4.9.1	Assumption manifest	82
4.9.2	One-use authorization	82
4.9.3	Runtime assurance monitor	82
4.9.4	Stopping distance	84
4.10	Counterexamples that teach the architecture	84
4.10.1	“The type says validated”	84
4.10.2	“The simulator found no gouge”	84
4.10.3	“The command starts with status”	85
4.10.4	“Preflight passed a moment ago”	85
4.10.5	“Upload timed out, so retry”	85
4.10.6	“The preview looks right”	85
4.10.7	“The machine stopped because abort returned”	85

4.11 A practical architecture for Dropcut and the Z1	85
4.11.1 Suggested packages	86
4.11.2 Trust boundaries	86
4.11.3 Capability policies	86
4.12 Staged implementation roadmap	86
4.12.1 Stage 0: Establish artifact identity	86
4.12.2 Stage 1: Define reference semantics	87
4.12.3 Stage 2: Add translation validators	87
4.12.4 Stage 3: Build abstract machine analysis	87
4.12.5 Stage 4: Harden geometry checking	87
4.12.6 Stage 5: Certificate graph and policies	87
4.12.7 Stage 6: Controller state machine and runtime binding	88
4.12.8 Stage 7: Optimization with checked feasibility	88
4.13 Capstone: the complete recurring job argument	88
4.13.1 Source and elaboration	88
4.13.2 Intent and planning	89
4.13.3 Scheduling and machine lowering	89
4.13.4 Controller lowering	89
4.13.5 Runtime	89
4.13.6 Composed conclusion	90
4.14 Chapter summary	90
4.15 Exercises	90
4.15.1 Contracts and invariants	90
4.15.2 Abstract interpretation	90
4.15.3 Translation validation	91
4.15.4 Certificates	91
4.15.5 Controller protocol	91
4.15.6 Runtime assurance	91
4.15.7 Capstone implementation	91
Glossary	93
Selected Exercise Solutions	97
Chapter 1	97
Chapter 2	98
Chapter 3	98
Chapter 4	99
Design Checklists	100
Language and IR checklist	100
Geometry and planning checklist	100
Certificate and runtime checklist	100
Further Reading and References	102
Source note	103

Preface

A desktop CNC mill makes compiler theory unusually concrete. A compiler bug in a conventional program may corrupt data or crash a process. A compiler bug in a CAM system can move a sharp rotating tool through a fixture, destroy a workpiece, or damage the machine. Yet the software chain that produces CNC motion is often treated as a convenient script that prints G-code.

This book develops a different mental model:

Central idea. A CAM system is a compiler for a cyber-physical process. Its source language describes manufacturing intent. Its target language describes machine and controller actions. Correctness means that every permitted target execution implements the source intent within explicit geometric, numerical, and physical assumptions.

The book is organized as four cumulative chapters rather than a catalog of disconnected topics.

1. **Chapter 1 asks what the compiler means.** We begin with a small milling job and build the vocabulary of intent, state, paths, trajectories, stock, semantics, refinement, assumptions, and error.
2. **Chapter 2 asks how to represent and transform that meaning.** We design a JavaScript authoring language, a ladder of intermediate representations, composable path and command APIs, and explicit compiler-pass contracts.
3. **Chapter 3 asks how to construct good motions.** We study tool geometry, swept volumes, robust computation, pocket and surface planning, free-space linking, operation scheduling, and feed optimization.
4. **Chapter 4 asks why the result should be trusted.** We introduce assertions, invariants, abstract interpretation, translation validation, proof-carrying artifacts, final-byte checking, controller protocols, and runtime assurance.

Each important idea follows the same learning sequence:

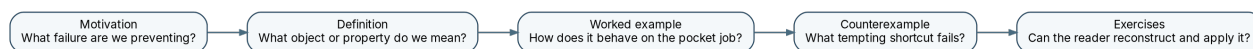


Figure 1: The learning sequence used throughout the book.

- **Motivation:** the concrete failure or design pressure that makes the idea necessary.
- **Definition:** a precise meaning and its boundary.
- **Worked examples:** usually applied to one recurring Makera Z1 job.
- **Counterexamples:** plausible shortcuts that fail.
- **Exercises:** reconstruction, calculation, design, and implementation tasks.

The intended reader can program and is comfortable with elementary algebra and vectors. Prior knowledge of compiler construction, formal methods, category theory, computational geometry, or operations

research is not assumed. Sidebars introduce the minimum foundation when one of those subjects first becomes useful.

The recurring job

Most examples use the same part so that abstractions accumulate instead of resetting. Throughout the book, an **artifact** means an immutable compiler input or output - source, geometry, IR, evidence, machine profile, or exact job bytes - identified independently of a transient filename or object address. The stock is an aluminum plate measuring 80 by 50 mm. The program must produce:

- a rectangular pocket 30 by 20 mm and 4 mm deep;
- a 4 mm through-hole;
- a protected exterior surface that must not be gouged;
- a fixture keep-out region occupying the rightmost 12 mm of the setup;
- a work coordinate system, G54, measured at the stock's upper-left corner;
- a 3.175 mm flat end mill, T1, and a 4 mm drill, T2.

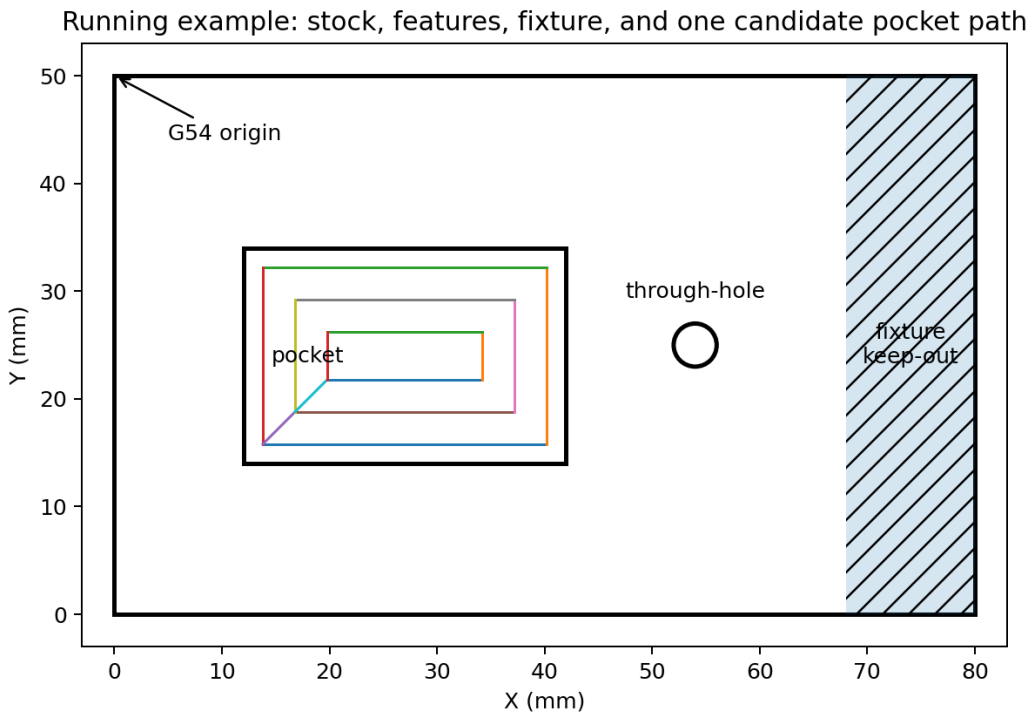


Figure 2: The running job used throughout the book.

This is deliberately simple enough to draw and calculate by hand. It is still rich enough to expose the central problems: frames, units, tool choice, stock evolution, entry motions, path ordering, collision checking, controller modality, probing, final-byte identity, and runtime preflight.

How to read the book

A reader building software should implement the running example after each chapter. A reader studying the theory can work the mathematical exercises without a machine. Actual cutting should begin only after independent dry-run, simulation, air-cut, and machine-specific safety procedures. The examples are archi-

tectural and pedagogical; they are not a substitute for manufacturer instructions, workholding judgment, feeds-and-speeds validation, or operator supervision.

Chapter 1

What Are We Compiling?

The hardest mistake to recover from is made before any code is written: choosing the wrong thing as the meaning of the system. If G-code text is treated as the meaning, every higher-level idea becomes a comment or convention. If manufacturing intent and physical state are treated as the meaning, G-code becomes one target encoding among several.

By the end of this chapter, you should be able to:

- explain why CAM is a refinement compiler rather than a text generator;
- distinguish geometry, path, trajectory, machine action, controller syntax, and physical execution;
- describe a job using denotational, operational, axiomatic, and temporal semantics;
- state compiler correctness as a relation between allowed source outcomes and possible target executions;
- identify assumptions and uncertainty that remain outside software proof;
- formulate basic safety and conformance properties for the recurring pocket job.

1.1 Begin with the failure, not the file

Motivation. Why not define a CAM program as “whatever G-code it prints”?

Consider these two controller programs:

```
G0 Z10
G0 X12 Y14
G1 Z-1 F100
G1 X42 F400
```

```
G53 G0 Z-2
G54 G0 X12 Y14
G1 Z-1 F100
G1 X42 F400
```

They contain similar coordinates, but their meanings depend on modal state, coordinate systems, machine position, controller dialect, and whether the first move is interpreted in machine or work coordinates. Text alone does not tell us whether the tool crosses a fixture, whether the spindle is running, which tool is installed, or whether the cut removes required material.

Now consider a user request:

“Rough this pocket with T1, leave 0.2 mm radial and axial stock, then finish it without touching the protected exterior.”

That request leaves many choices open. A raster, offset spiral, adaptive clearing path, or trochoidal path may all be acceptable. A source language that names one exact polyline too early discards the useful freedom to choose among them. A source language that merely prints controller words never states what counts as acceptable.

Definition - manufacturing intent. A manufacturing intent is a specification of acceptable process outcomes and process constraints, independent of any one concrete toolpath or controller encoding.

For the running pocket, intent includes at least:

```
interface PocketIntent {
  region: PlanarRegion<"G54">;
  floorZ: Mm;
  roughingAllowance: { radial: Mm; axial: Mm };
  finishingTolerance: Mm;
  protectedMaterial: SolidRef;
  permittedTools: readonly ToolConstraint[];
}
```

The interface does not say “move to X=13.587.” It says which material may be removed, which must remain, and how closely the result must match the target.

Worked example. The pocket operation may denote every final stock solid that:

1. contains all protected target material;
2. contains no material that should have been removed beyond the allowed residual tolerance;
3. was produced using a permitted tool and process state;
4. respects fixture, travel, and setup constraints.

A planner chooses one witness from this set.

Counterexample - text equivalence. Two G-code files can differ textually but have the same interpreted motion behavior because one repeats modal words and the other omits them. Conversely, one character can change G90 to G91 and produce a radically different trace. Text equality is neither necessary nor sufficient for semantic equality.

1.1.1 The first design principle

The first design principle of the book is therefore:

Design rule. Make machining intent and physical effects the semantic model. Treat G-code as a target language whose meaning is supplied by an explicit controller interpreter.

This mirrors the useful separation in the NIST RS274 work: an interpreter maps modal RS274/NGC syntax into canonical machining functions [R1, R2]. A CAM compiler works primarily in the opposite direction. It begins with intent, constructs canonical operations, and eventually serializes them into a controller dialect.

1.2 A compiler for a physical process

Motivation. Ordinary compiler diagrams end at machine code. A CAM diagram must continue through a controller and a physical process.

Definition - compiler. A compiler is a program that translates an artifact in one language into an artifact in another language while preserving or refining a specified meaning.

Definition - artifact. An artifact is an immutable input or output of a compiler stage: source text, an AST, an IR program, geometry, a machine profile, evidence, or exact controller bytes. We use *artifact* rather than *file* because some artifacts are structured values or binary data, and because filenames do not identify content.

Fundamentals aside - functions and relations. A function assigns one output to each accepted input. A relation may associate one input with several possible outputs. Physical commands are often better modeled as relations because measurement, faults, and controller behavior can produce several possible successors. A refinement relation states which input-output pairs a compiler pass is allowed to produce.

The definition has three parts:

1. **Languages:** source and target artifacts have grammars and well-formedness rules.
2. **Meaning:** each artifact denotes behaviors, values, outcomes, or traces.
3. **Correctness relation:** target meaning must be compatible with source meaning.

A JavaScript function that writes strings is not automatically a compiler in this stronger sense. It becomes one when the source and target languages, their semantics, and their translation relation are explicit.

Definition - cyber-physical system. A cyber-physical system combines discrete computation and communication with continuous physical state, sensors, actuators, timing, and environmental interaction.

A CNC system is cyber-physical because:

- controller commands are discrete;
- position, velocity, spindle speed, and material removal are continuous;
- probing introduces measured values and uncertainty;
- communication can time out or lose boundaries;
- alarms, interlocks, pause, resume, and abort form a concurrent protocol;
- machine and workpiece geometry constrain legal motion.

1.2.1 The semantic waist

A system with several source APIs and controller targets needs a stable middle.

Definition - semantic waist. A semantic waist is a compact intermediate language that preserves the domain distinctions required by all frontends and backends while hiding source syntax and target encoding details.

For CAM, useful canonical actions include:

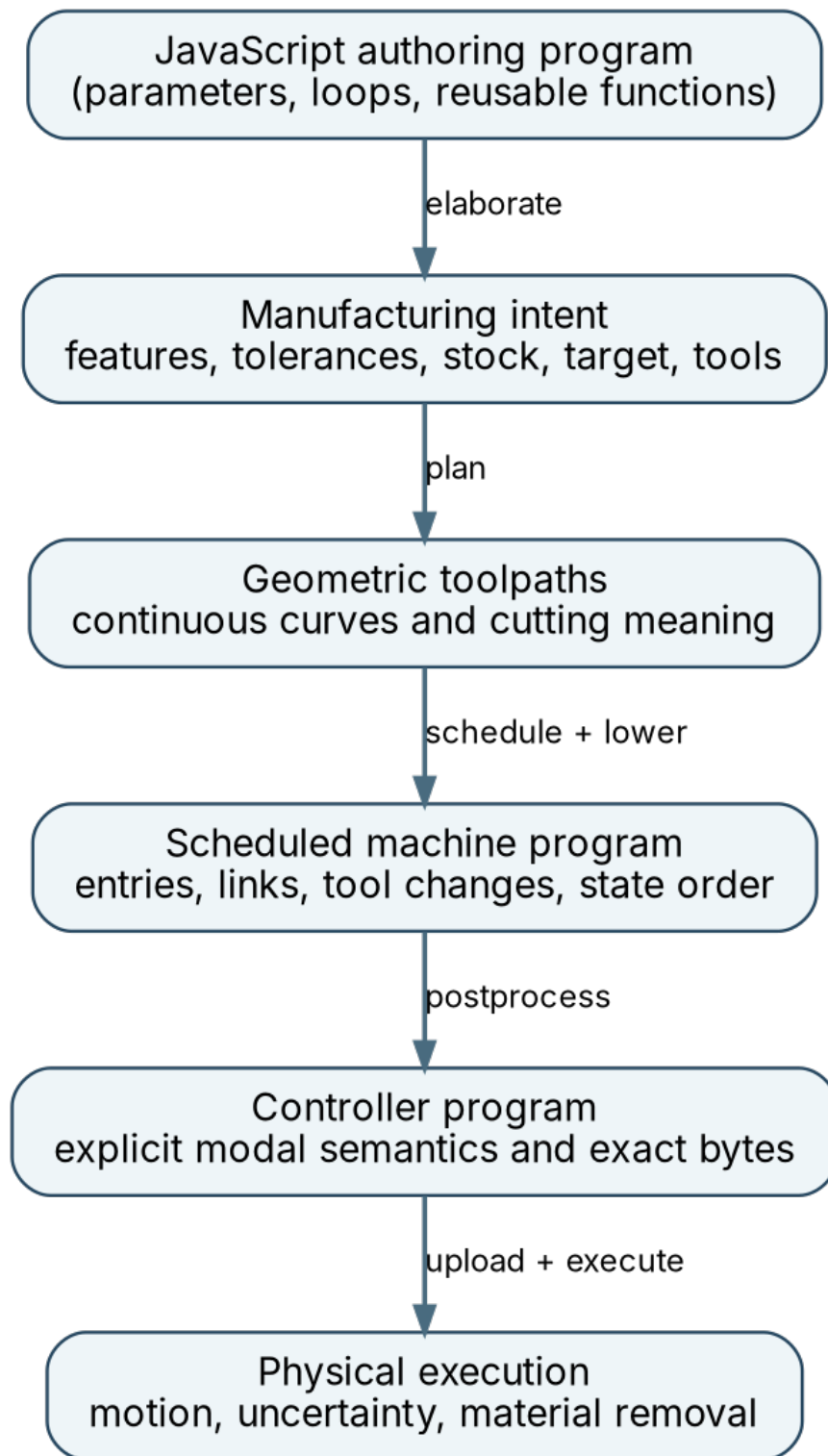


Figure 1.1: A CAM compiler continues beyond text into physical execution.

```

type CanonicalCommand =
  | { kind: "selectTool"; tool: ToolRef }
  | { kind: "startSpindle"; rpm: Rpm; direction: "cw" | "ccw" }
  | { kind: "stopSpindle" }
  | { kind: "traverse"; path: Path; clearance: ClearanceRequirement }
  | { kind: "cut"; path: Path; tool: ToolRef; feed: FeedPolicy }
  | { kind: "probe"; path: Path; output: MeasurementId }
  | { kind: "dwell"; duration: Seconds };

```

Definition - canonical command. A canonical command is a controller-independent machine action whose physical and state effects are explicit. It is the semantic waist between planning and target dialects.

Definition - clearance requirement. A clearance requirement is a proposition that the moving tool assembly remains separated from specified stock, target, fixture, and machine obstacles throughout a non-cutting motion.

There is no G0 in this definition. “Traverse safely from A to B” is the meaning. One backend may lower it to a single coordinated rapid. Another may retract Z, move XY, then descend. The backend is free to choose only among implementations that satisfy the traverse contract.

Worked example. On the running plate, moving from the hole to the pocket while the tool is at Z=-1 mm is not a safe traverse because the current stock occupies the space between them. The canonical operation carries a clearance requirement. The Z1 backend may implement it as:

```

retract to certified clearance plane
move in XY
descend to entry height

```

That sequence is a refinement of the abstract traverse.

Fundamentals aside - abstraction. An abstraction deliberately forgets details that do not matter at one level. The abstract traverse forgets whether the controller uses one or three blocks. It does not forget that the motion must be non-cutting and collision-free. Good abstraction removes accidental detail while retaining proof-relevant meaning.

1.3 The physical vocabulary: solids, frames, paths, and stock

Formal language becomes useful only after the physical nouns are clear.

1.3.1 Solids

Motivation. Statements such as “do not hit the fixture” require a mathematical object for both the moving assembly and the fixture.

Definition - solid. In the ideal model, a solid is a subset of three-dimensional space:

$$S \subseteq \mathbb{R}^3.$$

A tool, holder, stock blank, target part, and fixture are different solids. A mesh, boundary representation, signed-distance field, voxel grid, and dixel grid are different data structures that approximate or encode

solids. A **signed-distance field** stores the signed distance to a boundary; a **voxel grid** divides space into occupied or empty cells; a **boundary representation (B-rep)** stores faces, edges, vertices, and their incidence relationships. Chapter 3 compares their proof strengths.

A key distinction is between:

- **stock** S_0 : material present before cutting;
- **target** P : material intended to remain;
- **protected material** $P_{protected}$: material the tool must not enter beyond tolerance;
- **fixture/obstacles** O : geometry that must not be contacted;
- **cutting tool** T_c : the portion intended to remove material;
- **tool assembly** T_a : cutter, shank, collet, nut, and holder used for collision checking.

Counterexample - cutter-only collision checking. A short end mill can clear a pocket wall while the collet nut strikes the stock or fixture. Proving that T_c is clear does not prove that T_a is clear.

1.3.2 Frames

Motivation. The number (10, 20, -2) is meaningless until we know whether it is expressed in machine coordinates, G54 work coordinates, part coordinates, or a local feature frame.

Definition - coordinate frame. A coordinate frame gives an origin and basis in which coordinates are interpreted. A rigid transform between frames belongs to the group $SE(3)$ of orientation-preserving rigid motions.

For a three-axis mill with fixed tool orientation, most transforms are translations and perhaps a setup rotation about Z. Designing with general rigid transforms prevents the API from assuming that all jobs remain axis aligned.

```
interface Point3<F extends FrameId> {
    x: Mm;
    y: Mm;
    z: Mm;
    frame: F;
}

interface RigidTransform<A extends FrameId, B extends FrameId> {
    from: A;
    to: B;
    translation: Vec3<Mm>;
    rotation: Quaternion;
}
```

Fundamentals aside - quaternions. A quaternion is a four-number representation of three-dimensional rotation. It avoids the singularities of Euler angles and composes efficiently. The API need not expose quaternion algebra to ordinary users, but the frame artifact must define one unambiguous rotation representation.

The type parameters make an illegal mix visible:

```
const pMachine: Point3<"machine"> = ...;
const pG54: Point3<"G54"> = ...;
```



```
// Reject: points from different frames cannot be subtracted directly.
const delta = subtract(pMachine, pG54);
```

Worked example. The pocket origin is (12, 14, 0) in G54. If the measured G54 origin in machine coordinates is (105.2, 72.4, -18.0), then the machine-space pocket origin is obtained by the frame transform. If the work offset changes, every geometric certificate depending on that transform becomes stale.

1.3.3 Paths and trajectories

Motivation. A line drawn through space does not state how fast the machine follows it, whether it cuts, or whether the controller follows the same geometric interpolation.

Definition - geometric path. A path is a continuous map from a normalized parameter interval into configuration space:

$$\gamma : [0, 1] \rightarrow Q.$$

Definition - configuration space. Configuration space Q is the set of all complete poses or joint configurations relevant to the moving mechanism. A path is a curve through this space.

For fixed-orientation three-axis milling, Q can often be approximated as $\mathbb{R}^3$. For multi-axis machining, Q also includes orientation or joint coordinates.

Definition - trajectory. A trajectory is a time-indexed configuration:

$$q : [0, T] \rightarrow Q.$$

A monotone path parameterization $s : [0, T] \rightarrow [0, 1]$ turns a path into a trajectory:

$$q(t) = \gamma(s(t)).$$

The path determines geometry. The time law determines velocity, acceleration, jerk, and cycle time.

Worked example. The pocket perimeter can be the same geometric rectangle at 100 mm/min or 600 mm/min. These are the same path but different trajectories and potentially different cutting processes.

Counterexample - feed as path metadata only. If an optimizer shortens a corner radius but leaves the same feed, the required centripetal acceleration increases. Geometry and timing interact; a feed number cannot always be validated independently of path curvature.

1.3.4 Material-removal state

Definition - ideal subtractive update. If a cutting motion removes volume R_i , the ideal stock update is:

$$S_{i+1} = S_i \setminus R_i.$$

Definition - swept volume. The swept volume of a solid is the union of every position occupied by that solid along a trajectory. It converts a moving-geometry question into a set in space.

For a tool solid T_c and trajectory $X(t)$:

$$\text{Sweep}(T_c, X) = \bigcup_{t \in [0, T]} X(t)T_c.$$

An ideal cut may set $R_i = \text{Sweep}(T_c, X)$ intersected with current stock. Real cutting involves deflection, runout, dynamics, and material effects; those enter as uncertainty or a richer process model.

Invariant - stock monotonicity. In a purely subtractive model:

$$S_{i+1} \subseteq S_i.$$

Traverse, dwell, spindle change, and tool selection leave stock unchanged. This simple distinction is already useful for detecting semantic bugs.

1.4 Manufacturing intent denotes a set, not one path

Motivation. Why should a pocket operation have many acceptable implementations?

Suppose the target pocket floor is at $Z=-4$ mm with a finishing tolerance of 0.03 mm. The user does not care whether roughing uses parallel passes or an offset spiral, provided the final result, safety constraints, and process requirements are satisfied.

Definition - denotation of intent. The denotation of an intent I , written $\llbracket I \rrbracket$, is the set of executions or outcomes allowed by that intent.

For a simplified pocket:

$$\llbracket I_{\text{pocket}} \rrbracket = \{S' \mid \text{ProtectedIntact}(S') \wedge \text{ResidualWithinTolerance}(S') \wedge \text{RequiredFeaturesPresent}(S')\}.$$

The set can also constrain traces, resources, or intermediate states. For example, it may require roughing before finishing or prohibit cutting above a maximum engagement.

1.4.1 Planner as witness constructor

Definition - witness. A witness is a concrete construction supplied to demonstrate that an existential claim is true.

The intent says, in effect:

$$\exists P. \text{Implements}(P, I).$$

A planner returns one candidate P and supporting data:

```
interface PlanningResult<W> {
    toolpath: ToolpathProgram;
    witness: W;
    diagnostics: readonly Diagnostic[];
}
```

An independent checker evaluates the relation:

```
checkPlanningWitness(intent, result.toolpath, result.witness)
```

This producer-checker separation will become central in Chapter 4.

Worked example. A rectangular-pocket planner can return offset contours plus a **coverage witness** - checkable data showing how required material is covered - that partitions the required region into cells, each reached by the guaranteed inner sweep of at least one pass. The checker need not trust how the contours were discovered; it verifies coverage and no-gouge relations.

1.4.2 Permitted nondeterminism

Intent often admits choices:

- tool selection from a compatible set;
- entry location;
- path orientation;
- roughing pattern;
- operation ordering among independent operations;
- feed schedule within limits.

This is **permitted nondeterminism**: the source intentionally leaves choices to the compiler. Correct compilation narrows the set of behaviors but must not add forbidden ones.

Definition - machine profile. A machine profile is a versioned artifact describing target capabilities and limits: axis travel, kinematics, supported interpolation, feed and spindle ranges, acceleration assumptions, tools, probing, and controller compatibility. It is part of compilation input, not an informal global setting.

Counterexample - accidental overspecification. If the source API stores one exact raster before the machine profile is known, a later backend cannot choose a safer direction that avoids a fixture or respects axis dynamics. The IR has committed before it possesses the information needed to choose well.

1.5 Four complementary semantics

One semantic style rarely answers every question in a cyber-physical compiler.

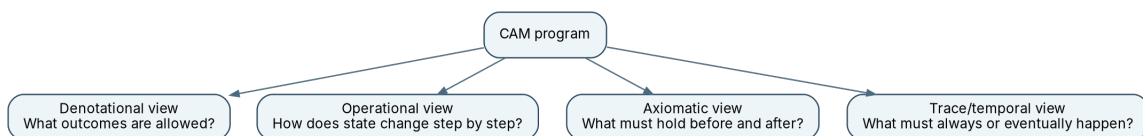


Figure 1.2: Four semantic views answer different questions.

1.5.1 Denotational semantics: what does it mean?

Definition - denotational semantics. A denotational semantics maps syntax to a mathematical domain in a compositional way.

Examples:

- a region expression denotes a subset of a plane;
- a path expression denotes a curve;
- a pocket intent denotes acceptable residual stock states;
- a canonical program denotes a relation over machine and stock states.

Compositionality means that the meaning of a compound object is computed from the meanings of its parts.

For sequential commands $c_1; c_2$ interpreted as relations:

$$\llbracket c_1; c_2 \rrbracket = \llbracket c_1 \rrbracket; \llbracket c_2 \rrbracket.$$

1.5.2 Operational semantics: how does it run?

Definition - operational semantics. An operational semantics defines execution through transitions between configurations.

A small-step rule has the shape:

$$\langle c, \sigma \rangle \rightarrow \langle c', \sigma' \rangle.$$

A big-step rule summarizes termination:

$$\langle c, \sigma \rangle \Downarrow (\sigma', e).$$

Here σ is machine/process state and e is an event or observation.

A simplified cut rule can be written:

$$\frac{\text{ReadyToCut}(\sigma, T, f, \gamma)}{\langle \text{Cut}(\gamma, T, f), \sigma \rangle \Downarrow (\sigma[q := \text{end}(\gamma), S := S \setminus \text{Sweep}(T, \gamma)], \text{CutTrace})}.$$

The premise gathers preconditions. The conclusion makes state change explicit.

1.5.3 Axiomatic semantics: what must be true before and after?

Definition - Hoare triple. A Hoare triple has the form:

$$\{P\} c \{Q\}.$$

It states that if precondition P holds and command c terminates normally, postcondition Q holds afterward [R4].

For the running job:

$$\{\text{Homed} \wedge \text{WCSKnown} \wedge \text{Tool} = T1 \wedge \text{SpindleOn} \wedge \text{PathSafe}(\gamma)\}$$

$$\text{Cut}(\gamma, T1, 400)$$

$$\{\text{Pose} = \text{end}(\gamma) \wedge S' = S \setminus \text{Sweep}(T1, \gamma)\}.$$

Hoare logic helps derive preflight requirements and local invariants. It does not by itself model network concurrency or continuous dynamics unless those are represented in the state and commands.

1.5.4 Trace and temporal semantics: what must always or eventually happen?

Definition - temporal semantics. Temporal semantics assigns meaning to complete traces and to properties that relate events across time, such as “always,” “eventually,” and “until.”

Definition - trace. A trace is a finite or infinite sequence of states and observable events generated by execution.

Definition - safety property. A safety property says that a bad event never occurs. A finite prefix demonstrates a violation.

Definition - liveness property. A liveness property says that a desired event eventually occurs, under stated fairness and environment assumptions.

Examples in temporal notation:

$$\Box(\text{Motion} \Rightarrow \text{Homed} \wedge \text{Authorized})$$

“Whenever motion occurs, the machine is homed and the motion was authorized.”

$$\Box(\text{AbortRequested} \Rightarrow \Diamond(\text{Stopped} \vee \text{Faulted}))$$

“An abort request eventually reaches an explicit stopped or faulted state.”

Temporal semantics becomes essential for upload, start, hold, resume, disconnect, alarm, and recovery. Lamport’s Temporal Logic of Actions is a standard foundation for such state-transition reasoning [R8].

1.5.5 Why all four are needed

Question	Best starting view
What final pocket shapes are acceptable?	Denotational
What does a probe command do step by step?	Operational
Which state is required before a cut?	Axiomatic
Can a lost acknowledgement lead to duplicate motion?	Trace/temporal

Counterexample - one simulator as semantics. A viewport that animates one nominal path provides a useful example execution. It does not define all behaviors, prove preconditions, or cover lost messages and uncertain measurements. Treating that renderer as “the semantics” silently erases the questions it cannot represent.

1.6 State: what the next command depends on

Motivation. Why can the same line of G-code behave differently at different times?

Because machine and controller state persist. A useful semantic state is larger than XYZ position:

$$\sigma = (q, \dot{q}, F, W, T, S_p, C, S_{stock}, P, O, M, t, A),$$

where:

- $q, \dot{q}$: positions and velocities;
- F : frame graph and uncertain transforms;
- W : active work coordinate system;
- T : active tool assembly;
- S_p : spindle/process state;
- C : coolant and accessories;
- S_{stock} : current stock;
- P : target/protected material;
- O : fixtures and obstacles;
- M : controller mode and modal state;
- t : time;
- A : alarms, interlocks, authorization, and protocol state.

Not every compiler pass needs every component. Each analysis can project the state to the properties it uses. Omitting a component is sound only if the pass proves it cannot affect the claimed relation.

1.6.1 Modal state

Definition - modal state. A modal setting remains active until replaced. In G-code, units, distance mode, plane, motion mode, feed, spindle speed, and work offset are common modal components.

A block containing only X10 may mean a rapid, linear feed, clockwise arc, or probing motion depending on prior state. Therefore, final-byte validation needs a modal interpreter and an explicit initial modal state.

Worked example. If the postprocessor assumes G90 but the controller begins in G91, the pocket coordinates become incremental. A correct job bundle either emits a preamble that establishes the required mode or names the initial-state assumption and verifies it at runtime. Emitting the preamble is usually safer.

1.6.2 Process state and stock state

Definition - stock state. A stock state is the compiler’s material model at one specific point in the scheduled execution, together with the artifact identity and enclosure guarantees of that model.

Stock state is path dependent. A low link between two cuts may be unsafe before roughing and safe after the material between them has been removed. Therefore, link checking must reference the stock state at the point in the schedule where the link occurs.

```
interface ScheduledStep {
  beforeStock: StockStateRef;
  command: CanonicalCommand;
  afterStock: StockStateRef;
}
```

Counterexample - checking all links against final stock. Final stock has less material than intermediate stock. A link proven clear only against final stock can pass through material that exists when the move actually runs.

1.7 Contracts, assertions, invariants, and assumptions

These words are often used loosely. We define them now because the rest of the book relies on their distinctions.

Definition - assertion. An assertion is a proposition about a specific program point, artifact, or state.

Example: “Before block 214, the spindle is definitely on.”

Definition - precondition. A precondition is a proposition required before an operation may execute or a theorem may be applied.

Example: cut requires a loaded tool, valid spindle state, known frame, positive feed, and a path satisfying its geometric obligations.

Definition - postcondition. A postcondition is promised after successful execution under the precondition and assumptions.

Definition - invariant. An invariant holds initially and is preserved by every relevant transition:

$$I(\sigma_0)$$

$$I(\sigma) \wedge \sigma \rightarrow \sigma' \Rightarrow I(\sigma').$$

Definition - assumption. An assumption is an external fact used by an argument but not established by that argument.

Examples of assumptions:

- the installed T1 diameter lies in [3.170, 3.180] mm;
- the fixture model matches the physical setup;
- the G54 measurement error is at most 0.015 mm;
- the controller follows the specified arc convention;
- machine following error is within a calibrated bound.

Definition - guarantee. A guarantee is a property established when its assumptions and preconditions hold.

1.7.1 Representation invariants versus physical guarantees

A PathBuilder can guarantee that its end field equals the endpoint derived from its final segment. That is a **representation invariant**. It does not guarantee that:

- the path avoids a fixture;
- the tool fits inside the pocket;
- the controller follows the same arc;
- the machine is homed;
- the physical tool diameter matches the model.

Counterexample - “type-safe means machine-safe.” Branded TypeScript units can prevent adding millimeters to RPM. They cannot inspect the spindle, measure the work offset, or prove a mesh matches reality. Type safety establishes properties of data and program construction, not the whole physical setup.

1.7.2 Weakest precondition intuition

Suppose the final requirement is:

```
spindle is off
machine is at safe park position
stock satisfies feature tolerance
```

Working backward through stop spindle, retract, and cut derives the conditions needed before the cut. This backward calculation is called weakest-precondition reasoning and will be developed in Chapter 4.

1.8 Correctness as refinement

Motivation. If the source intent allows many paths, target behavior cannot be required to equal one source trace. What should correctness mean instead?

Definition - refinement. A target artifact refines a source artifact when every observable target behavior is permitted by the source specification after applying the appropriate abstraction.

Let:

- I be source intent;
- B be the compiled job bundle;
- A be physical and runtime assumptions;
- $\text{Exec}(B, A)$ be possible target executions;
- α abstract low-level traces to intent-level observations.

Definition - abstraction function. An abstraction function maps detailed target traces to the observations relevant at the source level. For example, it may forget line numbers and acknowledgements while retaining tool poses, material removal, alarms, and final stock.

Definition - semantic preservation. Semantic preservation is the requirement that compilation retain source meaning according to a stated equality, refinement, or bounded-refinement relation.

A bounded correctness statement is:

$$\forall \tau \in \text{Exec}(B, A), \quad \alpha(\tau) \in N_\varepsilon(\llbracket I \rrbracket),$$

where N_ε is the allowed neighborhood under named metrics and tolerances.

This statement contains several important ideas:

1. **All target executions:** not one successful simulation.
2. **Explicit assumptions:** the theorem is conditional.
3. **Abstraction:** controller events are mapped to manufacturing observations.
4. **Bounded approximation:** numerical and physical deviation is quantified.
5. **Allowed source set:** the compiler may choose among valid implementations.

1.8.1 Exact preservation, refinement, and bounded refinement

Three pass relations recur:

Exact semantic preservation

$$\text{Sem}(O) = \text{Sem}(I).$$

Example: serializing with one deterministic canonical byte encoding, then parsing back, yields the same Controller IR.

Trace refinement

$$\text{Traces}(O) \subseteq \text{Traces}(I).$$

Example: an abstract traverse permits any safe connection; lowering chooses retract-XY-descend.

Bounded geometric refinement

Definition - Hausdorff distance. For compact sets A and B , the Hausdorff distance is the largest distance from a point in either set to its nearest point in the other. It measures worst-case set displacement, not timing or process equivalence.

$$d_H(\gamma_I, \gamma_O) \leq \varepsilon.$$

Example: an arc is linearized into chords with a proven maximum deviation.

Fundamentals aside - partial correctness of compilation. A correct compiler is allowed to reject an input it cannot compile soundly. CompCert’s semantic-preservation statement explicitly permits compile-time failure rather than incorrect code generation [R5]. In CAM, “cannot establish safety under this tolerance and setup” is a legitimate and often necessary result.

1.8.2 Composition across passes

If pass P_1 establishes relation $R_1(A, B)$ and pass P_2 establishes $R_2(B, C)$, the compiler needs a composition rule:

$$R_1(A, B) \wedge R_2(B, C) \Rightarrow R(A, C).$$

For exact equality, composition is simple. For error bounds, the rule must account for amplification. For stateful refinement, it may need a simulation relation. This is why pass contracts are part of semantics, not documentation.

1.9 Error, uncertainty, and model reality

Motivation. A mathematically exact proof about an inaccurate tool model can still produce an inaccurate part.

Three kinds of imperfection must be separated.

1.9.1 Numerical error

Numerical error arises from finite precision and approximation inside computation:

- floating-point rounding;
- curve sampling;
- mesh tessellation;
- arc fitting;
- coordinate formatting.

It can often be bounded by algorithms and checkers.

1.9.2 Model uncertainty

Model uncertainty is the gap between the ideal data model and the physical object:

- tool diameter and runout;
- stock dimensions;
- fixture placement;
- mesh-to-CAD deviation;
- work-offset measurement;
- machine calibration.

Some uncertainty is measured at runtime; some remains an operator assumption.

1.9.3 Process uncertainty

Process uncertainty arises during cutting:

- tool and workpiece deflection;
- thermal expansion;
- backlash or servo error;
- chip recutting;
- material variation;

- spindle-speed fluctuation.

A simple hobby CAM system may model these only through conservative allowances and empirical process limits. It should not silently describe them as mathematically proved.

Definition - deterministic error bound. A deterministic bound states:

$$d(x, \hat{x}) \leq \varepsilon$$

for every case covered by the assumptions.

Definition - probabilistic bound. A probabilistic statement has a confidence level:

$$\Pr(d(X, \hat{x}) \leq \varepsilon) \geq 1 - \alpha.$$

Definition - empirical observation. An empirical observation reports measured performance on tests. It is evidence about behavior, but it is not automatically a universal bound.

These categories should be explicit in certificates.

1.9.4 Inner and outer approximations

Definition - conservative approximation. A conservative approximation is deliberately biased so that accepting a check cannot hide a violation under the modeled assumptions. The direction of bias depends on the proposition.

Definition - inner and outer approximation. An inner approximation X^- is guaranteed to lie inside the true set; an outer approximation X^+ is guaranteed to contain it.

For an uncertain solid X , maintain:

$$X^- \subseteq X_{true} \subseteq X^+.$$

- Use **outer** tool sweeps and obstacle models to prove no collision.
- Use **inner** cutting sweeps to prove material was definitely removed.
- Use **outer** residual-stock models to prove no required material remains.

The direction matters. One sampled approximation rarely proves both no-gouge and complete removal.

Counterexample - one mesh for every claim. A mesh that lies inside the true fixture can miss collision. A mesh that lies outside the true target can report false gouges. Every geometric artifact needs an enclosure interpretation, not just a triangle count.

1.10 Worked derivation: the first semantic model of the pocket

We now describe the running job without choosing a toolpath.

Conservative geometry uses different enclosures for different claims

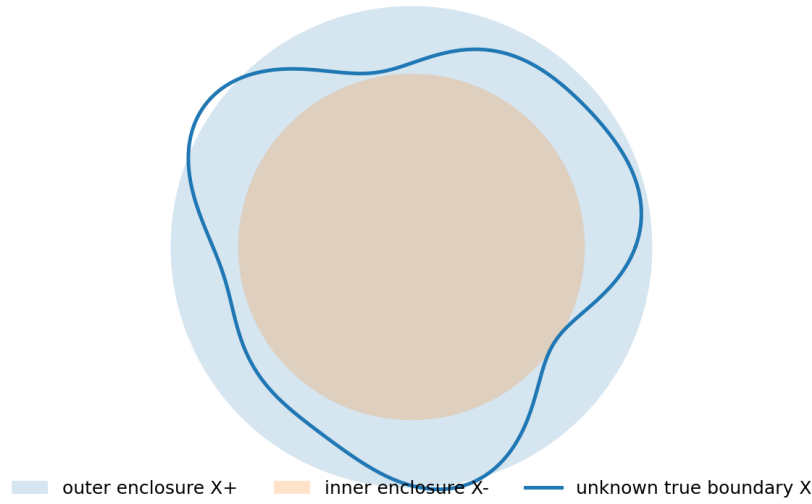


Figure 1.3: Inner and outer enclosures support different proof directions.

1.10.1 Inputs

```
interface PocketProject {
  setup: {
    stock: SolidRef;
    target: SolidRef;
    fixtures: readonly SolidRef[];
    frameGraph: FrameGraph;
  };
  tools: readonly ToolAssembly[];
  intent: readonly OperationIntent[];
  machine: MachineProfileRef;
}
```

Let:

- S_0 be the 80 by 50 by 8 mm stock;
- P be the target plate with pocket and hole removed;
- O be the right-side fixture keep-out solid;
- T_1 and T_2 be tool assemblies;
- $F_{G54 \rightarrow M}$ be the uncertain work-to-machine transform.

1.10.2 Source-level outcome predicate

A simplified final predicate is:

$$\text{GoodFinal}(S_f) \equiv P^- \subseteq S_f \wedge S_f \subseteq P^+ \wedge \text{HoleOpen}(S_f) \wedge O \text{ unchanged.}$$

The inner and outer target envelopes encode tolerance. This is only a geometric predicate. A complete intent may additionally require:

- approved tools;
- maximum depth per pass;

- rough-before-finish precedence;
- no cutting during traverse;
- final spindle-off and safe-park state.

1.10.3 Program-level trace predicate

For every execution trace τ :

$$\text{SafeTrace}(\tau) \equiv \forall t, \text{ToolAssembly}(t) \cap O = \emptyset \wedge q(t) \in Q_{\text{admissible}} \wedge (\text{Cutting}(t) \Rightarrow \text{SpindleValid}(t)).$$

1.10.4 End-to-end goal

The compiler’s aspirational theorem is:

$$\text{CheckBundle}(B) = \text{accept} \wedge \text{RuntimeAssumptionsHold}(A)$$

$$\Rightarrow$$

$$\forall \tau \in \text{Execute}(B, A), \text{SafeTrace}(\tau) \wedge \text{GoodFinal}(\text{FinalStock}(\tau)).$$

This theorem will not be obtained in one leap. Chapters 2 through 4 decompose it into representations, pass relations, geometric checks, static analyses, protocol invariants, and runtime assumption discharge.

1.11 Chapter summary

A CAM compiler should be designed around the following distinctions:

- **Intent is not a path.** It denotes acceptable outcomes and constraints.
- **A path is not a trajectory.** Time parameterization adds dynamics.
- **A path is not an action.** Cut, traverse, and probe have different effects.
- **A machine action is not G-code text.** Controller syntax is a target encoding with modal semantics.
- **A simulation is not a proof.** It demonstrates particular executions or samples.
- **A representation invariant is not a physical guarantee.** Types and builders constrain data, while runtime assumptions connect the model to reality.
- **Correctness is refinement under assumptions and error bounds.** It must cover all permitted target executions.

1.12 Exercises

1.12.1 Concept checks

1. Explain in your own words why “generate G-code” is an insufficient semantic specification for a CAM API.
2. Classify each object as intent, path, trajectory, action, controller syntax, or physical assumption:
 - “leave 0.2 mm stock”;
 - a circular arc from A to B;
 - G2 X20 Y10 I5 J0;

- a 400 mm/min time law;
 - “the installed tool diameter is within 0.005 mm of nominal”;
 - “probe toward -Z until contact.”
3. Give one safety property and one liveness property for pause/resume.
 4. Explain why final stock alone cannot express every valid process constraint.

1.12.2 Calculations

5. A path is a quarter circle of radius 10 mm. It is followed at constant path speed 600 mm/min. Compute its duration. Then compute the centripetal acceleration in mm/s².
6. A work-frame angular uncertainty is 0.0003 radians. Bound the positional contribution at a point 60 mm from the frame origin using the small-angle approximation.
7. Stock outer bounds are $[0, 80.02] \times [0, 50.02] \times [-8.03, 0.01]$ mm in G54. State whether the point (80.01, 25, -2) is definitely inside, definitely outside, or uncertain relative to nominal stock and relative to the outer enclosure.

1.12.3 Modeling

8. Write a structured intent for a 6 mm drilled hole with a 0.05 mm positional tolerance and a protected annulus around it.
9. Extend the state tuple with one component needed for automatic tool changing and explain which transitions use it.
10. Write an operational rule for a traverse that preserves stock.
11. Write a Hoare triple for probing the stock top and installing a measured G54 Z origin.
12. State an end-to-end refinement theorem for an attended air cut, where no stock removal is expected.

1.12.4 Counterexample construction

13. Construct two G-code prefixes after which the block X10 has different meanings.
14. Give a case where two operations produce the same final stock but are not semantically interchangeable.
15. Give a case where cutter collision checking succeeds but full assembly collision checking fails.
16. Give a case where a correct mathematical path still produces an unsafe physical result because an assumption is false.

1.12.5 Implementation project

17. Define TypeScript types for SolidRef, FrameId, Point3<F>, Path<F>, and CanonicalCommand that make units and frames explicit.
18. Implement a tiny pure interpreter for selectTool, startSpindle, stopSpindle, traverse, and cut. Model stock as a set of named regions rather than real geometry. Add tests for stock monotonicity and “cut implies spindle on.”
19. Write a one-page semantic specification of the recurring pocket job without mentioning G-code.

Chapter 2

Designing the Language, IR, and Compiler Passes

Chapter 1 described what a CAM compiler should mean. We now turn that meaning into software architecture. The goal is not merely a clean API. The goal is an architecture in which every stage has a vocabulary, every transformation has a contract, and every loss of information is deliberate.

By the end of this chapter, you should be able to:

- use JavaScript as a staged authoring language without making arbitrary JavaScript the stable semantics;
- design dimensional and frame-safe values;
- divide the compiler into several intermediate representations with explicit legality rules;
- use path composition, effectful commands, and state tokens to expose invariants;
- classify compiler passes by the relation they must preserve;
- design provenance, hashing, diagnostics, and final-byte validation;
- map these ideas onto the Dropcut Studio architecture.

2.1 Why a pleasant API needs an austere core

Motivation. Users want loops, functions, reusable geometry, parameters, and familiar editors. Checkers want finite, immutable, serializable data. How can one system support both?

JavaScript is a good authoring language for parametric work:

```
for (let i = 0; i < 4; i++) {  
  drill({  
    at: p(mm(10 + i * 12), mm(38), mm(0), "G54"),  
    diameter: mm(4),  
    depth: mm(8.5),  
  });  
}
```

But an unrestricted JavaScript closure is a poor long-lived IR node. It may read time, randomness, files, network state, global prototypes, or mutable objects. It may produce different plans on different runs. It cannot be content-addressed without defining its whole execution environment.

Definition - authoring language. The authoring language is the language in which a user conveniently constructs a job. It may include general computation, syntactic sugar, macros, and libraries.

Definition - object language. The object language is the inert, explicit language produced by authoring and consumed by later compiler passes.

Definition - schema. A schema is a versioned structural contract for serialized data: required fields, field types, variants, and validation rules. A schema establishes shape, not full semantic correctness.

Definition - deterministic and reproducible. An evaluation is deterministic when the same explicit inputs produce the same result. It is reproducible when another conforming environment can reconstruct that result from the recorded source, versions, inputs, and seed. Determinism is a property of behavior; reproducibility also requires complete records and stable encodings.

The central boundary is:

```
JavaScript source + declared inputs
      |
      | isolated, deterministic evaluation
      v
immutable Authoring AST
      |
      | elaboration
      v
well-typed Plan IR
```

This is a form of **staging**.

Definition - staging. Staging separates computation into phases. An earlier phase executes to construct code or data for a later phase. Multi-stage programming makes this phase distinction explicit [R9].

Definition - abstract syntax tree (AST). An AST is a structured representation of source constructs after parsing, without preserving irrelevant punctuation. In this system, the staged Authoring AST is data constructed by JavaScript rather than executable JavaScript itself.

Definition - source span. A source span identifies a range in a content-addressed source artifact so diagnostics and provenance can point to the exact authoring text that created a node.

The JavaScript phase may compute a list of holes. The later compiler never calls the original loop or closure. It receives data such as:

```
interface DrillNode {
  kind: "drill";
  id: OperationId;
  at: Point3<"G54">;
  diameter: Mm;
  depth: Mm;
  source: SourceSpan;
}
```


2.1.1 The inert-AST rule

Design rule. After script evaluation, the compiler should retain no user closure, promise, proxy, prototype-dependent object, or ambient capability. It should retain only schema-validated immutable data and provenance.

A useful evaluation record is:

```
interface ScriptEvaluationRecord {
  sourceHash: Hash;
  languageVersion: string;
  apiVersion: string;
  declaredInputs: readonly ArtifactRef[];
  deterministicSeed?: bigint;
  resultAstHash: Hash;
  diagnostics: readonly Diagnostic[];
}
```

The record states exactly what would be required to reproduce the AST.

Worked example. The running job's JavaScript can declare stock and features with convenient helpers. Evaluation produces a JSON-like AST containing the stock dimensions, G54 frame name, pocket node, hole node, tool declarations, source spans, and explicit numeric values. A headless compiler can then build the same Plan IR without loading the editor or re-running JavaScript.

2.1.2 Isolation and termination

Definition - capability. A capability is an explicit object or token granting access to a specific operation. Passing only the capabilities a script needs reduces ambient authority and documents the permitted interface.

A capability object improves API discipline:

```
interface CamCapabilities {
  units: UnitConstructors;
  geometry: GeometryConstructors;
  tools: ToolRegistryBuilder;
  plan: PlanBuilder;
  diagnostics: DiagnosticSink;
}
```

However, passing capabilities into `new Function` in the same JavaScript realm is not a security boundary. The script still has access to standard constructors and can loop forever.

A production host should use a separately terminable worker, process, or language isolate with:

- no ambient filesystem or network access unless granted;
- memory and wall-clock limits enforced outside the script;
- deterministic time and randomness;
- structured input and output serialization;
- versioned API modules;
- cancellation by terminating the worker;
- no shared mutable object graph with trusted compiler code.

Counterexample - post-hoc timeout. Code that runs the script and then checks `elapsed > limit` does not enforce a timeout. An infinite loop prevents the check from ever executing.

2.1.3 Scope combinators and asynchronous leakage

An authoring API may offer:

```
withTool("T1", () => {
  withFeed(mmPerMin(400), () => pocket(feature));
});
```

A synchronous implementation saves state, invokes the callback, and restores state in `finally`. If the callback returns a promise, restoration can happen before awaited work continues.

Counterexample - lexical scope that is not lexical. If `withTool` is synchronous but its callback performs `await loadMesh()`, operations after the `await` may observe the previous tool. The API appears scoped while the semantics are time-dependent.

Three sound options are:

1. reject asynchronous callbacks;
2. make the scope combinator explicitly `async` and await completion;
3. elaborate scopes immediately into immutable explicit context and prohibit dynamic ambient state.

The third option is usually simplest for a reproducible compiler.

2.2 Units and frames as part of the type system

Motivation. Numeric JavaScript values do not reveal whether 400 means millimeters per minute, inches per minute, RPM, or a coordinate. A point does not reveal its frame.

Definition - dimensional type. A dimensional type associates a numeric value with a physical dimension, such as length, angle, time, speed, or spindle rate.

```
type Mm = number & { readonly __unit: "mm" };
type Seconds = number & { readonly __unit: "s" };
type MmPerMin = number & { readonly __unit: "mm/min" };
type Rpm = number & { readonly __unit: "rpm" };
```

Branding is not runtime dimensional analysis, but it moves common mistakes to API boundaries.

```
function linearFeed(value: MmPerMin): FeedPolicy;
function spindleSpeed(value: Rpm): SpindlePolicy;

linearFeed(rpm(12000)); // static error
```

2.2.1 Constructors validate domains

A branded value should be created only through checked constructors:

```
function mm(x: number): Mm {
  if (!Number.isFinite(x)) throw new RangeError("length must be finite");
```

```

return x as Mm;
}

function positiveMm(x: number): Mm {
  if (!(x > 0) || !Number.isFinite(x)) {
    throw new RangeError("length must be finite and positive");
  }
  return x as Mm;
}

```

Different domains need different constructors. Zero is valid for a coordinate and invalid for tool diameter. Negative values may be valid Z coordinates but not depths when depth is modeled as magnitude.

Definition - refinement type. A refinement type is a base type restricted by a predicate, such as finite numbers greater than zero. TypeScript brands approximate refinement types when constructors enforce the predicate and unsafe casts are controlled.

2.2.2 Frame-parameterized geometry

```

interface Point3<F extends FrameId> {
  readonly x: Mm;
  readonly y: Mm;
  readonly z: Mm;
  readonly frame: F;
}

function transformPoint<A extends FrameId, B extends FrameId>(
  transform: RigidTransform<A, B>,
  point: Point3<A>,
): Point3<B>;

```

Frame identifiers should be semantic identities, not just labels. A frame graph checker verifies:

- transforms connect declared frames;
- no contradictory cycles exist beyond uncertainty;
- every operation references a reachable frame;
- transforms are rigid unless a different transform class is explicitly allowed;
- uncertainty is carried through composition.

Worked example. The pocket region is stored in feature:pocket-1. That local frame is translated by (12, 14, 0) into G54. G54 is measured into machine coordinates. The machine-lowering pass composes the transforms and propagates uncertainty. The source geometry remains stable even when the setup moves.

Counterexample - unit conversion at printing time. If internal values are untyped and the postprocessor alone decides whether they represent inches or millimeters, geometric analyses may run under one interpretation while the controller receives another. Units must be resolved before analysis.

2.3 Why several intermediate representations are necessary

Motivation. One universal IR either contains everything and guarantees nothing, or lowers too early and loses source meaning.

An intermediate representation is not merely a convenient object type.

Definition - intermediate representation (IR). An IR is a language used between compiler stages, with a syntax, well-formedness conditions, and intended semantics appropriate to its abstraction level.

MLIR is a prominent example of compiler infrastructure built around extensible representations and progressive lowering across abstraction levels [R7]. STEP-NC provides a manufacturing-specific precedent for retaining features, working steps, and process information above axis commands [R3]. A CAM compiler benefits from a ladder of IRs:

2.3.1 Authoring AST

The Authoring AST retains source concepts and source locations. It may still contain unresolved names and defaults because good diagnostics refer to the user’s vocabulary.

Legal Authoring AST conditions include:

- every node matches a versioned schema;
- no executable closures or cyclic object graphs remain;
- numeric values are finite;
- source spans refer to content-addressed source artifacts.

2.3.2 Elaborated Plan IR

Elaboration resolves:

- units;
- frames;
- tool names;
- geometry references;
- defaults and scoped settings;
- identifiers;
- source-level sugar.

Definition - elaboration. Elaboration translates convenient surface syntax into a more explicit representation, resolving contextual information and reporting errors while source structure is still available.

The Plan IR should be serializable and compilable independently of JavaScript.

2.3.3 Intent IR

Intent IR represents features and process requirements:

```
interface OperationIntent {
  id: OperationId;
  feature: FeatureRef;
  regions: readonly RegionRequirement[];
```

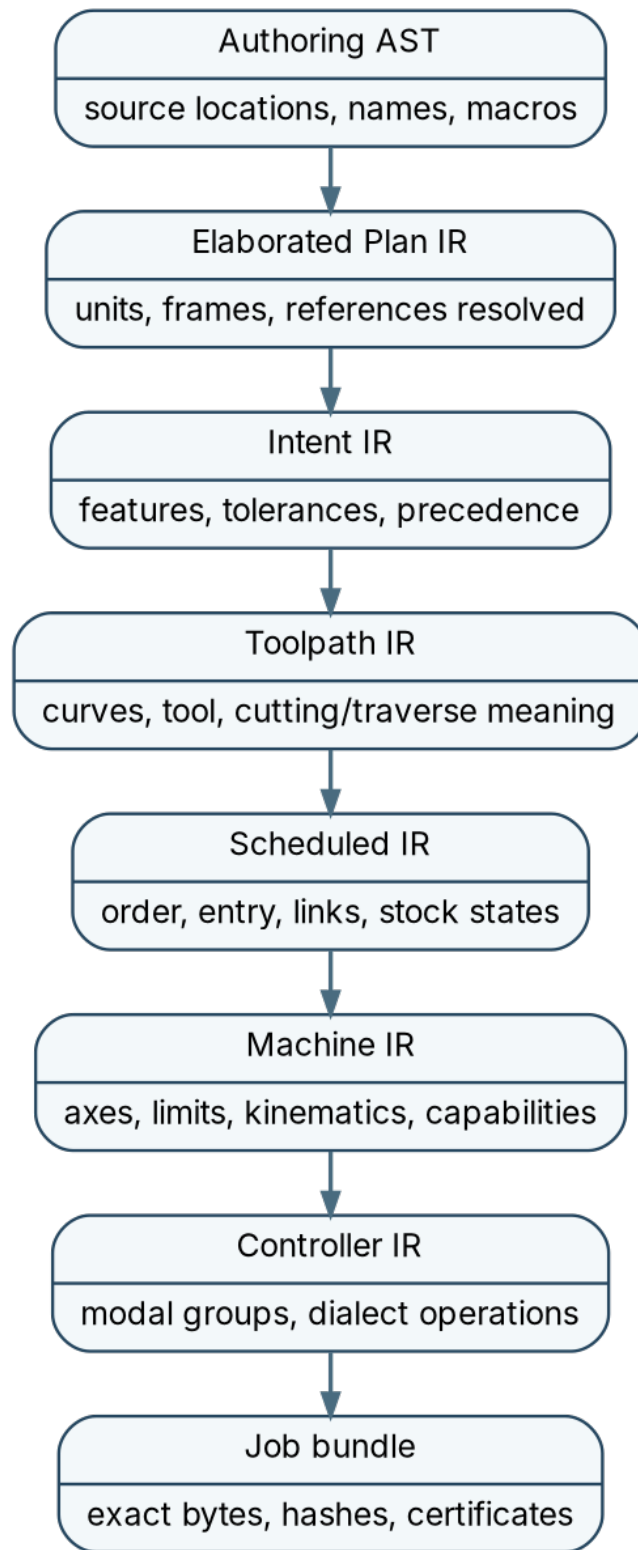


Figure 2.1: A multi-level IR ladder preserves information until the right decision point.

```
tolerances: readonly ToleranceRequirement[];  
toolConstraints: readonly ToolConstraint[];  
predecessors: readonly OperationId[];  
provenance: Provenance;  
}
```

It excludes exact tool-center curves.

2.3.4 Toolpath IR

Toolpath IR contains geometric curves and action meaning:

```
interface PlannedCut<F extends FrameId> {  
  path: Path<F>;  
  operation: OperationId;  
  tool: ToolRef;  
  phase: "entry" | "rough" | "finish" | "exit";  
  directionality: "reversible" | "forwardOnly";  
  feedPolicy: FeedPolicy;  
}
```

It remains machine independent where practical. A path can contain a true arc or spline even if a particular controller later requires linearization.

2.3.5 Scheduled IR

Scheduled IR introduces order and evolving state:

- tool changes;
- entries and exits;
- free-space links;
- stock-state references;
- probe-result dependencies;
- path orientation;
- feed schedule;
- operation precedence.

2.3.6 Machine IR

Definition - Machine IR. Machine IR is the first representation in which target-machine frames, kinematics, capabilities, and limits are concrete, while controller text and accidental modal syntax remain absent.

Machine IR resolves target capabilities:

- axis and kinematic constraints;
- machine and work frames;
- supported interpolation primitives;
- travel, speed, acceleration, and spindle limits;
- tool-change and probing capabilities;
- safe lowering of abstract traverses.

2.3.7 Controller IR and job bundle

Definition - controller dialect. A controller dialect is a particular grammar and operational interpretation of blocks, words, modal groups, numeric conventions, and extensions.

Definition - Controller IR. Controller IR is a structured, dialect-specific program whose operations and modal effects are explicit enough to interpret independently before textual serialization.

Controller IR makes modal operations explicit but is still structured. Exact bytes are generated only after controller semantics and number formatting are fixed.

The job bundle includes:

- exact bytes and hash;
- machine and controller profile hashes;
- tool/holder, stock, target, fixture, and frame artifacts;
- certificates and assumptions;
- preview data bound to the same artifact graph.

2.3.8 Legality at each level

Definition - IR legality. The legality predicate of an IR defines which operations, types, references, capabilities, and unresolved constructs may appear at that stage.

Examples:

Plan IR legal:

```
all units and references resolved
all values finite
every frame exists
```

Toolpath IR legal:

```
every path structurally continuous
every cut names tool and operation provenance
no source-language callback remains
```

Machine IR legal:

```
every operation supported or soundly lowered
every coordinate resolved to an allowed machine frame
```

Controller IR legal:

```
initial modal state explicit
no unparsed raw block in a production-certifiable program
```

Counterexample - “validated” as one universal stage. A value branded `ValidatedProgram` may indicate only that one function ran. It does not say which properties were checked, which artifact hash they concern, or whether later passes invalidated them. Validation should produce property-specific evidence attached to specific artifacts.

2.4 Paths as a compositional algebra

Motivation. Toolpath code frequently concatenates arrays of points and repairs gaps implicitly. A better API should make composition conditions explicit and testable.

A path from pose A to pose B can be written:

$$p : A \rightarrow B.$$

A second path $q : B \rightarrow C$ composes with it:

$$q \circ p : A \rightarrow C.$$

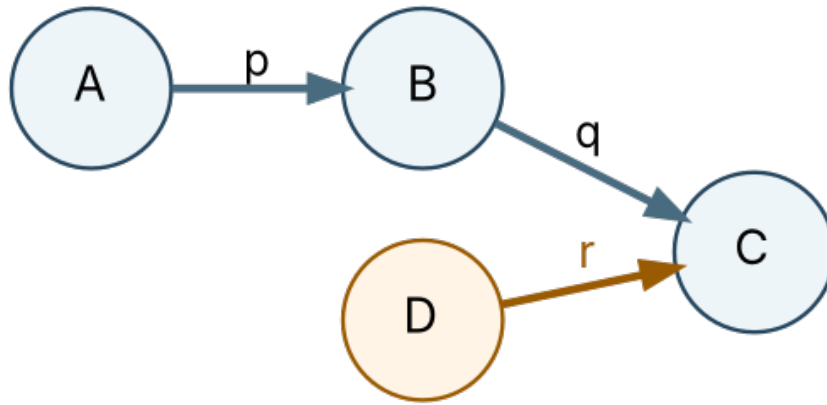


Figure 2.2: Paths compose when endpoints agree; the orange path begins at the wrong object.

Fundamentals aside - why mention category theory? Nothing in the implementation requires a category-theory library. The value is that a small set of laws tells us what composition should mean and supplies property tests. The mathematics is useful only insofar as it clarifies an engineering invariant.

Definition - category, minimal form. A category consists of objects, arrows between objects, identity arrows, and associative composition of compatible arrows.

For paths:

- objects are endpoint poses or exact point identities;
- arrows are paths;
- `emptyPath(A)` is the identity at A;
- concatenation is composition.

The laws are:

$$p \circ \text{id}_A = p$$

$$\text{id}_B \circ p = p$$

$$r \circ (q \circ p) = (r \circ q) \circ p.$$

The benefit is practical: the laws become property tests for builders, serializers, and transformations.

```
interface Path<F extends FrameId> {
  readonly frame: F;
  readonly start: Point3<F>;
  readonly end: Point3<F>;
  readonly segments: readonly Segment<F>[];
}

function concat<F extends FrameId>(
  left: Path<F>,
  right: Path<F>,
): Result<Path<F>, JoinError>;
```

2.4.1 Path image versus parameterization

Two curves can trace the same geometric image with different parameterizations. Concatenation changes the allocation of $[0, 1]$ between segments. Therefore, equality of function values is too strong. A path API should state whether semantic equality means:

- identical segment representation;
- equal image modulo monotone reparameterization;
- bounded Hausdorff distance;
- equal controller trace after lowering.

Each relation serves a different pass.

2.4.2 Approximate equality is not equality

Suppose joins accept endpoints when:

$$d(a, b) < \varepsilon.$$

This relation is not transitive. Let points lie on a line at 0 , 0.75 epsilon , and 1.5 epsilon . The first is close to the second, and the second is close to the third, but the first is not close to the third.

Counterexample - epsilon as object identity. If a contour sticher uses a tolerance-based packed key, repeated joins can merge unrelated contours or accumulate drift. Approximate proximity cannot silently replace exact topological identity.

Safer choices include:

- exact symbolic endpoint IDs;
- canonical snapping with recorded displacement;
- grid-edge IDs for marching algorithms;
- explicit join witnesses;
- accumulated error bounds.

```
interface JoinWitness<F extends FrameId> {
    leftEnd: Point3<F>;
    rightStart: Point3<F>;
    separation: Mm;
    action: "exact" | "snap" | "insertTraverse";
}
```

An inserted segment must be classified. Inserting a cutting line across a gap is not a numerical repair; it changes material removal.

2.4.3 Reversal is semantic

A point list may reverse mechanically, but a cut may be forward-only because of:

- climb/conventional direction;
- lead-in/lead-out shape;
- helical entry;
- probe direction;
- one-way compensation or process constraint.

The type should record directionality so an optimizer cannot reverse solely to reduce travel.

2.5 A path is not an action: effects enter the language

Motivation. The same path can mean rapid traverse, cutting feed, probe motion, or inspection scan. Geometry alone does not determine state change.

A command reads state, can fail, changes state, emits events, may produce a value, and consumes time. A simple semantic type is:

$$M(A) = \Sigma \rightarrow \text{Result}(A \times \Sigma, E).$$

```
type Command<A> = (
    state: MachineState,
) => Result<{ value: A; state: MachineState }, CommandError>;
```

A probe returns a measurement. A cut usually returns no user value but changes pose and stock.

Definition - effect. An effect is an interaction beyond pure value calculation: state mutation, failure, measurement, material removal, time, logging, or external I/O. Two expressions with the same returned value can differ semantically because their effects differ.

2.5.1 Why ordinary function composition fails

Let:

$$f : A \rightarrow M(B), \quad g : B \rightarrow M(C).$$

Ordinary composition does not type-check because f returns an effectful computation, not a bare B .

Definition - monad, operational intuition. A monad provides operations for placing a pure value into an effectful context and sequencing effectful computations while propagating their context [R10].

For state plus error:

```
function bind<A, B>(
  ma: Command<A>,
  f: (a: A) => Command<B>,
): Command<B> {
  return state0 => {
    const ra = ma(state0);
    if (!ra.ok) return ra;
    return f(ra.value.value)(ra.value.state);
  };
}
```

Definition - Kleisli composition. Kleisli composition is composition of functions of shape $A \rightarrow M$ using the monad's sequencing operation.

This is useful language for machine commands because it explains why the output state and result of one command become the input to the next.

Worked example - probing. A probe returns a measured top Z. The next command installs a work offset computed from that value:

```
const establishG54 = probeTop(surfaceRegion)
  .flatMap(measurement => installG54FromTop(measurement));
```

A plain list [probe, installOffset] does not express which measurement the second command consumes.

2.5.2 Indexed effects and typestate

A plain monad carries one state type. Machine protocols change which operations are legal.

Definition - typestate. Typestate encodes protocol states in types so operations are available only when the value is in an appropriate state [R11].

Definition - parameterized command. A parameterized or indexed command names its pre-state and post-state types:

$$\text{Cmd}\langle S_{\text{before}}, S_{\text{after}}, A \rangle.$$

```
type StartSpindle<S extends Homed & HasTool> =
  Cmd<S, S & SpindleRunning, void>;

type Cut<S extends Homed & HasTool & SpindleRunning & KnownWcs> =
  Cmd<S, S & AtPathEnd, void>;
```

Parameterized monads formalize effects whose state type changes [R12]. The concept is useful even if TypeScript cannot enforce every law.

Counterexample - types as live truth. A value typed `Homed` can be stale after a controller reset. Type-state constrains program construction; runtime preflight must re-establish live machine state.

2.6 An SSA-style state token

Motivation. Advanced type indices can become cumbersome in a TypeScript IR. We still need explicit ordering and dependency.

Static single assignment (SSA) gives every value one definition and makes data dependencies visible [R13]. The same idea can thread a machine-state token through effectful operations.



Figure 2.3: A state token makes effect order explicit.

```

%s0 = machine.initial
%s1 = machine.require_homed %s0
%s2 = tool.select %s1 @T1
%s3 = spindle.start %s2 12000rpm
%s4 = motion.cut %s3 path=@pocket feed=400
%s5 = spindle.stop %s4
  
```

Definition - state token. A state token is an SSA value consumed and produced by state-changing IR operations. It represents dependency and sequencing, not necessarily a complete copy of runtime state.

The verifier checks:

- every token has one definition;
- a token is not duplicated into contradictory machine futures;
- required state facts hold at each operation;
- merge points reconcile incoming abstract states;
- terminal states satisfy shutdown policy.

Pure geometry needs no token:

```

%offset = path.offset %boundary by=1.5875mm
%reverse = path.reverse %offset
  
```

Effectful scheduling does:

```

%s1 = motion.traverse %s0 to=%offset.start
%s2 = motion.cut %s1 path=%offset
  
```

2.6.1 Branches and measurements

```

%m, %s1 = probe.toward %s0 direction=-Z max=10mm
switch %m.status:
  contact(%z) -> install_frame %s1 %z
  no_contact  -> abort %s1
  alarm       -> quarantine %s1
  
```

A merge is legal only if the state requirements of subsequent commands hold on every incoming path.

Counterexample - duplicated state token. If %s2 feeds two cuts independently, the IR describes two successors from one physical state without ordering. Both may claim to start from the same stock. A linear or single-use token check rejects the fork unless the language explicitly models nondeterministic branching.

2.7 Passes are relations, not functions with hopeful names

Motivation. A pass named optimize, lower, or validate says almost nothing about what it is allowed to change.

A conventional pass API is:

```
function pass(input: I): O;
```

A certifying pass is conceptually:

```
interface Pass<I, O, W> {
  id: string;
  version: string;
  transform(input: I, config: Config): Result<{
    output: O;
    witness: W;
  }, Diagnostic[]>;
  check(input: I, output: O, witness: W, config: Config): CheckResult;
}
```

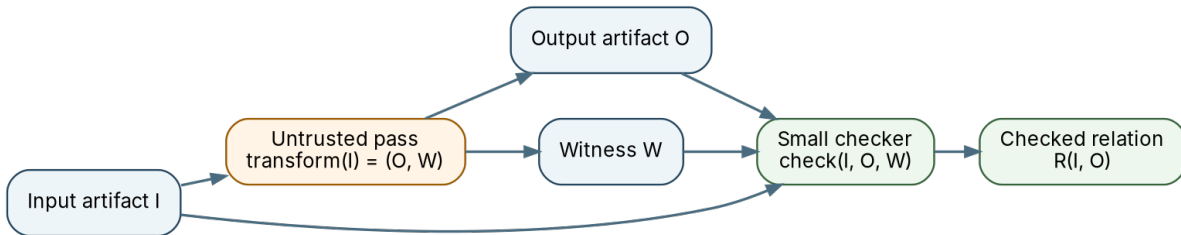


Figure 2.4: An untrusted pass proposes output and a witness; a checker establishes the relation.

Definition - pass relation. A pass relation $R(I, O)$ states exactly how the output is allowed to differ from the input.

2.7.1 Common pass relations

Canonicalization

$$\text{Sem}(O) = \text{Sem}(I).$$

Example: sorting commutative metadata or normalizing an explicit representation.

Refinement

$$\text{Beh}(O) \subseteq \text{Beh}(I).$$

Example: selecting one safe entry from several allowed entries.

Bounded approximation

$$d(\text{Sem}(I), \text{Sem}(O)) \leq \varepsilon.$$

Example: approximating a spline with arcs and lines.

Witness construction

$$O \in \llbracket I \rrbracket.$$

Example: a pocket path satisfies an intent predicate.

Optimization under preservation

$$\text{Feasible}(O) \wedge R(I, O) \wedge J(O) \leq J(I).$$

Example: reordering independent operations to reduce transition time.

2.7.2 Pass manifest

```
interface PassManifest {  
  id: string;  
  version: string;  
  inputSchema: SchemaId;  
  outputSchema: SchemaId;  
  relation: RelationId;  
  configHash: Hash;  
  determinism: "deterministic" | "seeded";  
  checker: CheckerId;  
  errorRule?: ErrorCompositionRule;  
}
```

The manifest belongs in provenance and cache keys.

2.7.3 Translation validation

Definition - translation validation. Instead of proving a transformer correct for every possible input, translation validation checks that one actual output correctly implements one actual input [R14].

This is attractive for a TypeScript CAM system. Toolpath planners, optimizers, and postprocessors can remain ordinary software. Independent checkers validate each produced artifact.

Good early candidates include:

- arc linearization;
- coordinate rounding;
- traverse expansion;
- modal compression;

- feed clamping;
- serialization and parse-back;
- operation reordering with precedence constraints.

Counterexample - validating before the last transform. If a program is checked, then the postprocessor rounds coordinates, inserts a preamble, linearizes arcs, and compresses modal words, the exact emitted program is not the artifact that was checked. Every semantics-changing or approximating pass after validation needs a relation and evidence.

2.8 Provenance and content identity

Motivation. When block 813 violates a clearance check, the user needs to know which source operation, strategy, and pass created it. When an input changes, stale evidence must be invalidated.

The source-span definition from Section 2.1 is now used as one component of provenance.

Definition - provenance. Provenance records the origin and transformation history of an artifact or sub-artifact.

```
interface Provenance {
  source?: SourceSpan;
  feature?: FeatureId;
  operation?: OperationId;
  parents: readonly Hash[];
  passHistory: readonly PassRecord[];
}
```

Synthetic commands need provenance too:

```
{
  reason: "safety-retract",
  introducedBy: "lower-traverse@2.1.0",
  parentMotions: ["sha256:..."]
}
```

2.8.1 Content addressing

Definition - content addressing. Content addressing identifies an artifact by a digest of its canonical content rather than by a mutable location or filename.

Definition - cryptographic hash. A cryptographic hash maps arbitrary bytes to a fixed-size digest designed so that accidental changes are detected and finding distinct inputs with the same digest is computationally difficult. The digest identifies bytes under the chosen algorithm; it does not prove their meaning.

Definition - canonical serialization. Canonical serialization is a deterministic rule that gives every semantic artifact exactly one byte encoding for hashing and comparison. Ordinary JSON is not canonical until key order, numbers, Unicode, missing fields, and versions are specified.

Definition - content-addressed artifact. A content-addressed artifact is identified by a cryptographic hash of canonical bytes and relevant schema information.

A filename is not identity. The same `part.nc` can contain different programs; the same bytes can have different filenames.

Canonical serialization must specify:

- field order;
- number representation;
- negative zero;
- Unicode normalization;
- omitted versus null fields;
- schema version;
- referenced artifact hashes.

```
interface ArtifactRef<T> {
    hash: Hash;
    schema: SchemaId;
    mediaType: string;
    byteLength: number;
    _type?: T;
}
```

Worked example. The pocket-path cache key includes the intent IR hash, target and stock hashes, T1 assembly hash, strategy version, tolerance budget, machine-relevant constraints, and deterministic seed. Omitting the tool hash could reuse a path generated for another diameter.

2.8.2 Diagnostics as structured artifacts

```
interface Diagnostic {
    code: string;
    severity: "info" | "warning" | "error";
    message: string;
    provenance?: Provenance;
    quantitative?: Record<string, number>;
    counterexample?: ArtifactRef;
    remediation?: string;
}
```

Stable codes allow tests and UI behavior without parsing prose.

2.9 The postprocessor is a compiler backend

Motivation. Formatting G-code looks simple until modality, arc conventions, precision, line boundaries, dialect extensions, and controller lifecycle are considered.

A postprocessor should lower structured Controller IR to exact bytes. It has at least four responsibilities:

1. establish an explicit initial modal state;
2. encode operations in the target dialect;
3. choose numeric formatting under an error budget;

4. emit a safe preamble, epilogue, and program lifecycle.

Definition - final-byte validation. Final-byte validation checks the exact serialized bytes that will be transferred and executed, rather than stopping at an earlier IR. It includes independent parsing, modal interpretation, numeric-bound checking, and content hashing.

2.9.1 Modal compression

Suppose Controller IR contains explicit operations:

```
set absolute distance
set millimeter units
linear move X=10 Y=0 Z=5 feed=400
linear move X=20 Y=0 Z=5 feed=400
```

A compressed target may omit repeated G1 and F400. Correctness is not textual equality. It is equality of interpreted canonical traces under the declared initial modal state.

Definition - parse-back validation. Parse-back validation independently parses emitted bytes, interprets their modal semantics, and compares the resulting trace with the pre-serialization Controller IR.

```
const bytes = serialize(controllerProgram);
const parsed = independentParser.parse(bytes, dialect);
const targetTrace = interpret(parsed, initialModalState);
const sourceTrace = interpret(controllerProgram, initialModalState);
checkTraceRelation(sourceTrace, targetTrace, roundingBudget);
```

2.9.2 Numeric formatting

Rounding from 12.34567 to 12.346 changes geometry. The postprocessor must charge this change to a compatible error budget. For independent coordinate rounding to precision ρ in three dimensions, a conservative Euclidean endpoint bound is:

$$\varepsilon_{round} \leq \frac{\sqrt{3}}{2} \rho,$$

where ρ is the decimal step. Path-interior and arc-center effects may require stronger analysis.

2.9.3 Raw commands

An escape hatch may be necessary:

```
interface RawControllerBlock {
  text: string;
  dialect: DialectId;
  declaredEffects?: EffectSummary;
  provenance: Provenance;
}
```

The declared effects are not proof. If an independent parser cannot determine the real effects, affected abstract state becomes unknown and production certification should fail closed.

Counterexample - first-token classification. A generic command path that classifies only the first token can approve `status\nG0 X100` as read-only. Authorization must parse the complete payload according to a closed grammar and classify every effect.

2.10 Worked compilation of the recurring job

We now trace the pocket through the IR ladder.

2.10.1 Authoring source

```
const stock = box({ x: mm(80), y: mm(50), z: mm(8) });
const T1 = flatEndMill({ diameter: mm(3.175), fluteLength: mm(12) });
const T2 = drillTool({ diameter: mm(4), fluteLength: mm(15) });

project({ stock, frame: "G54" }, () => {
  pocket({
    origin: p(mm(12), mm(14), mm(0), "G54"),
    size: v2(mm(30), mm(20)),
    depth: mm(4),
    roughingAllowance: mm(0.2),
    tool: T1,
  });

  drill({
    at: p(mm(54), mm(25), mm(0), "G54"),
    depth: mm(8.5),
    tool: T2,
  });
});
```

2.10.2 Elaborated Plan IR

The compiler resolves tool IDs, frame references, depth convention, defaults, and source spans:

```
{
  operations: [
    {
      id: "op:pocket-rough",
      kind: "pocket",
      frame: "G54",
      boundary: "artifact:rect-30x20-at-12x14",
      floorZ: -4,
      radialAllowance: 0.2,
      axialAllowance: 0.2,
      tool: "tool:T1"
    },
    {
      id: "op:drill-hole",
      kind: "drill",
      center: [54, 25, 0],
      depth: 8.5,
      tool: "tool:T2"
    }
  ]
}
```

```
]
}
```

2.10.3 Intent and toolpath IR

Intent normalization may split rough and finish operations, adding precedence:

```
rough-pocket -> finish-pocket
probe-top    -> rough-pocket
probe-top    -> drill-hole
```

The pocket planner proposes offset contours. The drill planner proposes a rapid-to-clearance, approach, drill cycle, and retract as semantic motions rather than raw G-code.

2.10.4 Scheduled and Machine IR

The scheduler groups T1 operations before the T2 tool change if dependencies allow. Link generation checks current stock. Machine lowering resolves G54 uncertainty, travel limits, and Z1 capabilities.

2.10.5 Controller IR and bytes

The backend emits explicit operations, then serializes:

```
G21 G17 G90
G54
T1 M6
S12000 M3
G0 Z10.000
G0 X13.588 Y15.588
G1 Z-1.000 F100.0
...
M5
G0 Z10.000
M30
```

The exact file is parsed back and compared with Controller IR. The job bundle binds its bytes to the tool, setup, machine profile, frame assumptions, and claims.

2.11 Applying the architecture to Dropcut Studio

The supplied Dropcut code already contains several useful separations:

- @cam/ir distinguishes paths and canonical commands;
- @cam/compiler lowers and validates programs;
- @cam/planner introduces entries, links, and refinement;
- @cam/strategies separates planning algorithms;
- @cam/post-maker and @cam/post-rs274 isolate target backends;
- @cam/analysis contains simulation and certificate structures;
- @cam/script-host provides a JavaScript authoring surface.

A pedagogically clean next architecture would sharpen the contracts:

```
script-host
  -> immutable Authoring AST

elaborator
  -> typed Plan IR

intent normalizer
  -> explicit Intent IR

strategy registry
  -> Toolpath IR + planning witness

independent geometry checker
  -> path/target/coverage claims

scheduler + linker
  -> Scheduled IR + feasibility witness

machine lowering
  -> Machine IR + capability/travel claims

Makera backend
  -> Controller IR

serializer + independent parser
  -> exact bytes + parse-back claim

runtime client
  -> hash-bound execution instance
```

The largest conceptual improvement is to separate **artifact** from **evidence about artifact**. A Validated-Program brand can remain a local API gate, but property-specific claims should live in a certificate graph keyed by hashes.

2.12 Chapter summary

The language and IR architecture should obey these principles:

- JavaScript is a staged macro language, not the trusted semantic core.
- Units, frames, finite values, and domain constraints are explicit before planning.
- Several IR levels retain the information needed by later decisions.
- Each IR has a legality predicate.
- Paths compose algebraically, but approximate joins require explicit witnesses.
- Actions are effectful and stateful; geometry alone is insufficient.
- Indexed commands or SSA state tokens expose legal sequencing.
- Every pass declares a semantic relation and, where practical, emits a witness checked independently.
- Provenance and content hashes bind transformations and diagnostics to exact artifacts.
- The postprocessor is a semantics-preserving or bounded-refinement backend whose exact bytes are parsed back and checked.

2.13 Exercises

2.13.1 Authoring language

1. Design an inert AST for a bolt-circle macro. Which values must be captured after JavaScript evaluation?
2. List six ambient JavaScript capabilities that harm reproducibility.
3. Write a protocol between an editor and a worker process that enforces cancellation and structured output.
4. Explain why freezing the outer object is insufficient if it contains mutable nested arrays.
5. Rewrite a dynamic `withTool` API into an explicit immutable context API.

2.13.2 Units and frames

6. Define branded types for angle, angular speed, acceleration, and jerk.
7. Decide whether `Depth` should be a signed coordinate or a positive magnitude in your API. Give advantages and failure modes of each design.
8. Write a frame-graph validation algorithm and state what happens when two paths imply inconsistent transforms.
9. Propagate a translation interval and a small angular interval through a point 40 mm from the frame origin.

2.13.3 IR design

10. Place each item in the earliest IR where it should appear: G17, source span, roughing allowance, exact tool-center arc, retract sequence, machine-axis limit, path directionality, decimal precision.
11. Write legality predicates for Intent IR and Machine IR.
12. Give an example of information lost by lowering too early.
13. Give an example of source detail that should not survive elaboration.
14. Design a versioning policy for IR schemas and pass relations.

2.13.4 Path algebra and effects

15. Prove identity and associativity for exact segment-list concatenation.
16. Construct three points that show non-transitivity of an epsilon join.
17. Design a `JoinWitness` checker.
18. Give four reasons a machining path may not be reversible.
19. Implement `pure` and `bind` for state plus error. Test the monad laws on finite sample states.
20. Model `probe`, `installWcs`, and `cut` using indexed command types.
21. Translate an imperative command list into SSA state-token form and detect one illegal reorder.

2.13.5 Compiler passes

22. Write a pass contract for arc linearization, including endpoint, sweep, and maximum-deviation obligations.
23. Write a pass contract for path reordering under precedence constraints.
24. Explain the difference between proving a transformer correct and translation-validating one result.
25. Design a witness for modal compression.
26. Design a cache key for constant-scallop planning and explain every field.
27. Show how a changed tool assembly hash invalidates downstream artifacts.

2.13.6 Project

28. Implement four TypeScript interfaces: `PassManifest`, `Certified<T>`, `Provenance`, and `ArtifactRef<T>`.
29. Build a tiny IR pipeline for the recurring job. Serialize every stage to canonical JSON and hash it.
30. Implement a miniature controller IR, serializer, independent parser, and trace comparison for G21, G90, G0, G1, F, S, M3, and M5.
31. Add a deliberately unsafe raw block and demonstrate that the analysis becomes unknown or rejects the production policy.

Chapter 3

Planning Geometry, Motion, and Optimization

The first two chapters separated intent from implementation and defined the languages through which that implementation will pass. We now confront the computational problem: how does the compiler construct a path and schedule that actually remove the right material, avoid the wrong material, and use the machine effectively?

CAM planning is not one algorithm. It is a chain of related problems:

```
intent
-> choose tool and process
-> construct cutting paths
-> add entries and exits
-> order operations
-> connect them through free space
-> parameterize motion in time
-> verify geometry and machine feasibility
```

By the end of this chapter, you should be able to:

- formulate planning as constrained witness construction;
- define tool and holder swept volumes and use configuration-space reasoning;
- compare stock representations and understand what they can prove;
- distinguish robust topological predicates from approximate geometric constructions;
- design adaptive refinement and typed error budgets;
- derive and check a simple pocket strategy;
- understand the role and limits of distance fields, Eikonal solvers, and constant-scallop ideas;
- formulate scheduling, linking, and feed planning as operations-research problems with hard feasibility constraints.

3.1 Planning begins with a feasible set

Motivation. An optimizer that minimizes cycle time but is allowed to trade fixture collision against a penalty is solving the wrong problem.

A planning problem should be divided into:

1. **feasibility**: which candidates satisfy every hard requirement;
2. **quality**: which feasible candidate is preferred.

Definition - feasible set. The feasible set $\mathcal{F}$ contains every candidate satisfying the hard geometric, process, machine, and protocol constraints.

Definition - objective function. An objective function $J(x)$ assigns a cost to a feasible candidate, such as cycle time, rapid distance, tool changes, predicted wear, or energy.

The optimization problem is:

$$\min_{x \in \mathcal{F}} J(x).$$

Safety is not one weighted term in J . It defines $\mathcal{F}$.

For the running job, a candidate schedule is feasible only if:

- every required pocket and hole operation appears;
- roughing precedes finishing;
- probe-dependent operations follow the probe;
- each cut uses a compatible tool and process range;
- every cutting and traverse sweep is safe at the referenced stock state;
- machine travel, speed, acceleration, and controller capabilities are respected;
- final stock and final machine state satisfy policy.

A possible objective is:

$$J = T_{cycle} + \lambda_1 N_{toolchange} + \lambda_2 D_{rapid} + \lambda_3 J_{wear}.$$

Worked example. Grouping all T1 operations before changing to T2 may reduce tool-change time. It is legal only if the hole does not need to be drilled before pocket finishing for structural or process reasons. The optimizer proposes; a feasibility checker confirms dependencies and geometric effects.

Counterexample - penalties for hard safety. Suppose collision adds a cost of one million while cycle time is measured in seconds. A solver may still select a collision if every collision-free candidate costs more than one million by the model. Hard constraints must be categorically enforced.

3.2 From tool shape and path to swept volume

Motivation. Endpoint checks cannot tell whether the body of a tool crosses an obstacle between endpoints.

Definition - swept volume. For a solid T and a time-varying rigid pose $X(t)$, the swept volume is:

$$\text{Sweep}(T, X) = \bigcup_{t \in [0, T]} X(t)T.$$

Different claims use different solids:

- cutting sweep: $\text{Sweep}(T_c, X)$;
- assembly collision sweep: $\text{Sweep}(T_a, X)$;
- probe body sweep: probe assembly under probe trajectory;
- machine structure sweep: moving spindle or gantry components if modeled.

3.2.1 Three distinct geometric obligations

No target gouge

$$\text{Sweep}^+(T_c, X) \cap P_{\text{protected}}^- = \emptyset.$$

No fixture or holder collision

$$\text{Sweep}^+(T_a, X) \cap O^+ = \emptyset.$$

Guaranteed required removal

$$R_{\text{required}}^+ \subseteq \text{Sweep}^-(T_c, X).$$

The superscripts show enclosure direction. No-gouge and collision use outer moving geometry and conservative obstacles. Guaranteed removal uses an inner sweep.

Counterexample - one “gouge” flag. A stock simulator that receives only stock and toolpath can detect some overcut relative to the stock boundary or spoilboard. It cannot prove conformance to a protected target surface that it was never given.

3.2.2 Lines, arcs, and general curves

For a translating convex tool along a line segment, the sweep has a simple extrusion interpretation. For circular arcs, the sweep can often be bounded analytically. For splines or arbitrary sampled paths, a checker may recursively subdivide parameter intervals and enclose the tool pose over each interval.

A generic continuous checker is:

```
check(interval I):
  P = conservative pose enclosure of path over I
  W = conservative sweep enclosure of tool assembly under P

  if W is separated from obstacles:
    prove I safe
  else if an inner intersection is known:
    return collision counterexample over I
  else if refinement limit reached:
    return inconclusive over I
  else:
    split I and recurse
```

An inconclusive result is not failure of soundness. It means the current representation or budget cannot decide the property.

3.3 Configuration space makes linking intelligible

Motivation. Free-space linking seems like a special CAM trick until we reformulate moving-solid collision as ordinary path planning.

Definition - configuration space. Configuration space Q is the space of all complete configurations of a movable object or mechanism. A single point in Q represents one pose or joint state [R18].

For a fixed-orientation translating tool assembly T_a and obstacle O , forbidden tool-reference positions form a configuration obstacle.

Definition - Minkowski sum. For sets A and B , the Minkowski sum is:

$$A \oplus B = \{a + b \mid a \in A, b \in B\}.$$

Expanding an obstacle by the reflected tool collects every reference-point position at which the tool would overlap the obstacle.

Thus:

$$O_C = O \oplus (-T_a).$$

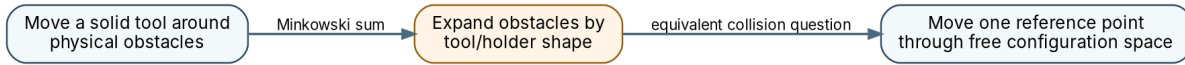


Figure 3.1: Configuration-space expansion turns solid motion into point-path planning.

A reference-point path γ is collision-free when:

$$\gamma([0, 1]) \cap O_C = \emptyset.$$

3.3.1 Why the current stock matters

At schedule step i , obstacles include fixtures and stock S_i . Material removed by earlier cuts changes configuration space:

$$O_{C,i} = (O \cup S_i) \oplus (-T_a).$$

The shortest safe link can therefore change as the job progresses.

Worked example. Before pocket roughing, a link across the pocket at $Z=-2$ mm intersects stock. After roughing removes the interior, the same link may be collision-free for the cutter but still unsafe for the holder near walls. A schedule-aware linker evaluates the correct S_i and full T_a .

3.3.2 Simple and advanced link policies

A conservative three-axis policy is:

```

retract vertically to a certified clearance Z
move in XY at that Z
descend vertically to entry height

```

This is easy to explain and check but may be slow. More advanced policies include:

- visibility graphs in a planar free-space slice;
- A* on a conservative occupancy grid;
- distance-field gradient paths;
- Eikonal shortest paths with position-dependent cost;
- multiple clearance planes;
- stock-aware low links when clearance is proved.

The abstract operation remains `Traverse`. The planner can improve its implementation without changing source intent.

Counterexample - “safe Z” as a magic scalar. A fixed safe Z is not safe if a tall fixture exceeds it, if machine Z travel cannot reach it with the current tool, or if work and machine frames are confused. A clearance plane is a geometric claim under a setup and tool assembly, not just a configuration value.

3.4 Stock representations and their proof power

Motivation. Stock simulation is often introduced as a rendering feature. Its representation determines which claims can be made and how expensive they are.

3.4.1 Height fields

A height field stores one Z value for each XY location:

$$h(x, y) = \text{highest occupied material at } (x, y).$$

It is efficient for top-down, three-axis jobs without overhangs. It cannot represent multiple disjoint intervals along Z or undercuts.

For the running pocket, a height field is adequate to represent ideal stock because each XY ray intersects remaining stock in one interval from the bottom upward.

3.4.2 Dexels and triple dexels

Definition - dixel. A dixel stores one or more occupied intervals along a sampling ray. A triple-dixel model uses three orthogonal ray families to improve surface representation [R20, R21].

Dexels update quickly under sweeps and can represent multiple intervals along a ray. Triple dexels reduce directional artifacts, but they remain discrete approximations whose enclosure properties must be specified.

3.4.3 Voxels and adaptive spatial trees

Definition - voxel and octree. A voxel is a three-dimensional occupancy cell. An octree recursively divides a cube into eight children, concentrating resolution near boundaries or uncertain regions.

Definition - topology. Topology describes connectivity, containment, boundaries, holes, and components without depending on exact distances. A tiny sign or key error can therefore cause a large topological change even when coordinate error is small.

Voxels and octrees can represent arbitrary topology, but memory and conservative boundary treatment matter.

A cell may be classified:

- definitely empty;
- definitely occupied;
- mixed/uncertain.

For collision proof, mixed cells are treated as occupied. For guaranteed removal, only definitely removed cells count.

3.4.4 Boundary representations and exact queries

Definition - boundary representation. A boundary representation, or B-rep, describes a solid through oriented faces, edges, vertices, and their topological incidence. Analytic surfaces can make design intent precise, while robust Boolean and sweep operations remain demanding.

A B-rep or analytic solid can support precise surface and intersection queries. Many practical systems combine:

- a mesh or B-rep as design geometry;
- acceleration structures for queries;
- a dixel/voxel representation for stock updates;
- conservative error bounds relating representations.

3.4.5 Choosing by claim

Representation	Strength	Limitation	Typical claim
Height field	Fast 3-axis updates	No undercuts/multi-layer	Residual top surface
Dixel	Multiple intervals	Directional sampling	Stock removal and interference
Triple dixel	Better boundary coverage	More memory and reconciliation	General 3-axis stock approximation
Voxel/octree	General topology	Resolution and memory	Conservative occupancy
B-rep/analytic	Precise design semantics	Robust Boolean complexity	Target and fixture definition

Design rule. Choose a representation and enclosure direction for a named proposition. Do not attach a generic “resolution” to the whole certificate.

3.5 Sampling is evidence only when connected to a theorem

Motivation. A path sampled every 0.2 mm can cross a 0.1 mm obstacle between samples.

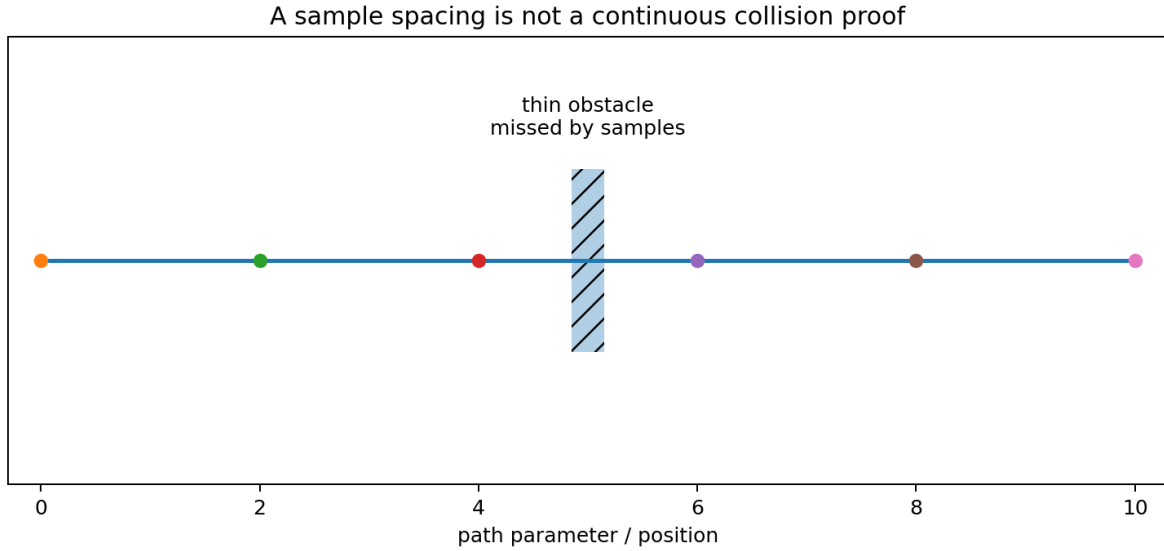


Figure 3.2: A thin obstacle can lie between every sampled point.

Definition - sampling. Sampling evaluates a continuous object at a finite set of parameter values. Sampling alone says nothing about unsampled locations.

To derive a continuous bound, one needs additional structure. Examples:

1. a Lipschitz bound on motion or distance;
2. a conservative cell occupancy rule;
3. interval evaluation over parameter ranges;
4. analytic extrema for primitive curves;
5. adaptive subdivision until separation is proved.

3.5.1 Lipschitz reasoning

Definition - Lipschitz constant. A function has Lipschitz constant L when output distance is never more than L times input distance. It is a worst-case sensitivity bound.

A function f is Lipschitz with constant L if:

$$\|f(x) - f(y)\| \leq L\|x - y\|.$$

If distance-to-obstacle is known to vary no faster than L , and a sample has clearance c , then a neighborhood of radius less than c/L is safe. The usefulness depends on obtaining a valid L and including tool/transform uncertainty.

3.5.2 Chord error for a circular arc

For an arc of radius R approximated by a chord subtending angle θ , the sagitta is:

$$e = R \left(1 - \cos \frac{\theta}{2} \right).$$

To ensure $e \leq \varepsilon$:

$$\theta \leq 2 \arccos \left(1 - \frac{\varepsilon}{R} \right).$$

This is a real continuous-domain bound, unlike a rule that merely samples at a nominal spacing.

Worked example. For $R = 10$ mm and $\varepsilon = 0.01$ mm:

$$\theta_{max} \approx 2 \arccos(0.999) \approx 0.08945 \text{ rad} \approx 5.13^\circ.$$

A quarter circle needs at least $\lceil 90/5.13 \rceil = 18$ chords under this bound.

3.5.3 Adaptive refinement

A sound refinement loop is:

```

propose local approximation
compute a certified bound for the claimed metric
if bound <= allocated budget:
    accept
else if subdivision remains possible:
    subdivide
else:
    return inconclusive

```

The output witness contains accepted cells or intervals, bounds, and unresolved regions.

Counterexample - endpoint-only arc validation. Two arcs can share endpoints while taking opposite sweeps or different centers. Endpoint equality does not preserve the path image or cutting effect.

3.6 Robust computational geometry

Motivation. A tiny numeric error in a coordinate may be harmless. A tiny error in a sign predicate can change topology: contours merge, polygon orientation flips, or an intersection disappears.

Definition - geometric predicate. A predicate answers a discrete geometric question such as orientation, sidedness, intersection, containment, or ordering.

Definition - robust predicate. A robust predicate returns the mathematically correct discrete answer for its stated numeric input model, including near-degenerate cases where ordinary floating point may choose the wrong sign.

Definition - geometric construction. A construction computes a coordinate or geometric object such as an intersection point, offset curve, or fitted arc.

For points a, b, c in 2D, orientation is the sign of:

$$\text{orient2d}(a, b, c) = (b_x - a_x)(c_y - a_y) - (b_y - a_y)(c_x - a_x).$$

Near collinearity, floating-point cancellation can return the wrong sign. Adaptive exact predicates evaluate quickly in ordinary cases and increase precision near degeneracy [R17].

3.6.1 Exact topology, bounded geometry

A useful design is:

- represent topological identity with exact symbolic or combinatorial identifiers;
- use robust predicates for connectivity decisions;
- use bounded approximate coordinates for constructions;
- record any snapping displacement in the error budget.

For a marching-squares contour, an endpoint can be identified by the exact grid edge it crosses rather than by packing rounded floating-point coordinates into a limited integer key.

Counterexample - periodic packed keys. If a coordinate is quantized and masked into a fixed number of bits, keys repeat periodically. Points separated by the wrap distance become identical to the topology algorithm. On a small desktop machine, the wrap distance can still lie inside the work envelope.

3.6.2 Interval arithmetic

Definition - interval enclosure. An interval $[a, b]$ represents every real value between its endpoints. Outward-rounded interval arithmetic guarantees that the exact result lies in the computed interval [R19].

If $x \in [a, b]$ and $y \in [c, d]$:

$$x + y \in [a + c, b + d].$$

Interval evaluation can bound:

- curve coordinates over a parameter interval;
- transform uncertainty;
- distance or implicit-surface values;
- polynomial extrema;
- sweep occupancy.

A wide interval can be subdivided. If it remains inconclusive, the checker rejects or reports unknown rather than guessing.

Fundamentals aside - the dependency problem. If $x \in [0, 1]$, naive interval arithmetic gives $x - x \in [-1, 1]$ even though the exact expression is zero. Repeated variables introduce overestimation. Symbolic simplification, subdivision, affine arithmetic, or Taylor models can improve precision without sacrificing enclosure.

3.7 Error budgets are typed proof objects

Motivation. Adding every tolerance into one scalar can combine incompatible quantities or ignore error amplification.

Definition - error budget. An error budget is a structured account of permitted approximation and uncertainty, together with rules that propagate component bounds to a final claim.

Possible metrics include:

```
type ErrorBound =
| { metric: "hausdorff-position"; frame: FrameId; value: Mm }
| { metric: "normal-surface"; surface: Hash; value: Mm }
| { metric: "max-gouge-depth"; target: Hash; value: Mm }
| { metric: "transform-translation"; transform: Hash; value: Mm }
| { metric: "transform-rotation"; transform: Hash; value: Radians }
| { metric: "axis-following"; axis: AxisId; value: Mm };
```

Only compatible metrics can be added directly.

3.7.1 Propagation through a pass

Definition - sensitivity. A sensitivity bound states how much a transformation can amplify input deviation. A Lipschitz constant is one common sensitivity measure.

If a transformation f has sensitivity L_f under a metric, input error ε_{in} and local approximation ε_f produce:

$$\varepsilon_{out} \leq L_f \varepsilon_{in} + \varepsilon_f.$$

A rigid transform has positional sensitivity one to translation error, but rotational uncertainty contributes a radius-dependent term. Near a topology change, an offset operation may not have a small global Lipschitz constant.

3.7.2 A sample budget

Suppose the final surface tolerance is 0.05 mm. A conservative allocation is:

Source	Bound
CAD/tessellation	0.008 mm
planning/refinement	0.012 mm
arc fitting	0.004 mm
output rounding	0.001 mm
frame/probing	0.010 mm
following error	0.010 mm
Total	0.045 mm

The remaining 0.005 mm is reserve. This sum is meaningful only if every component is a deterministic bound in a compatible position metric and no pass amplifies it beyond one.

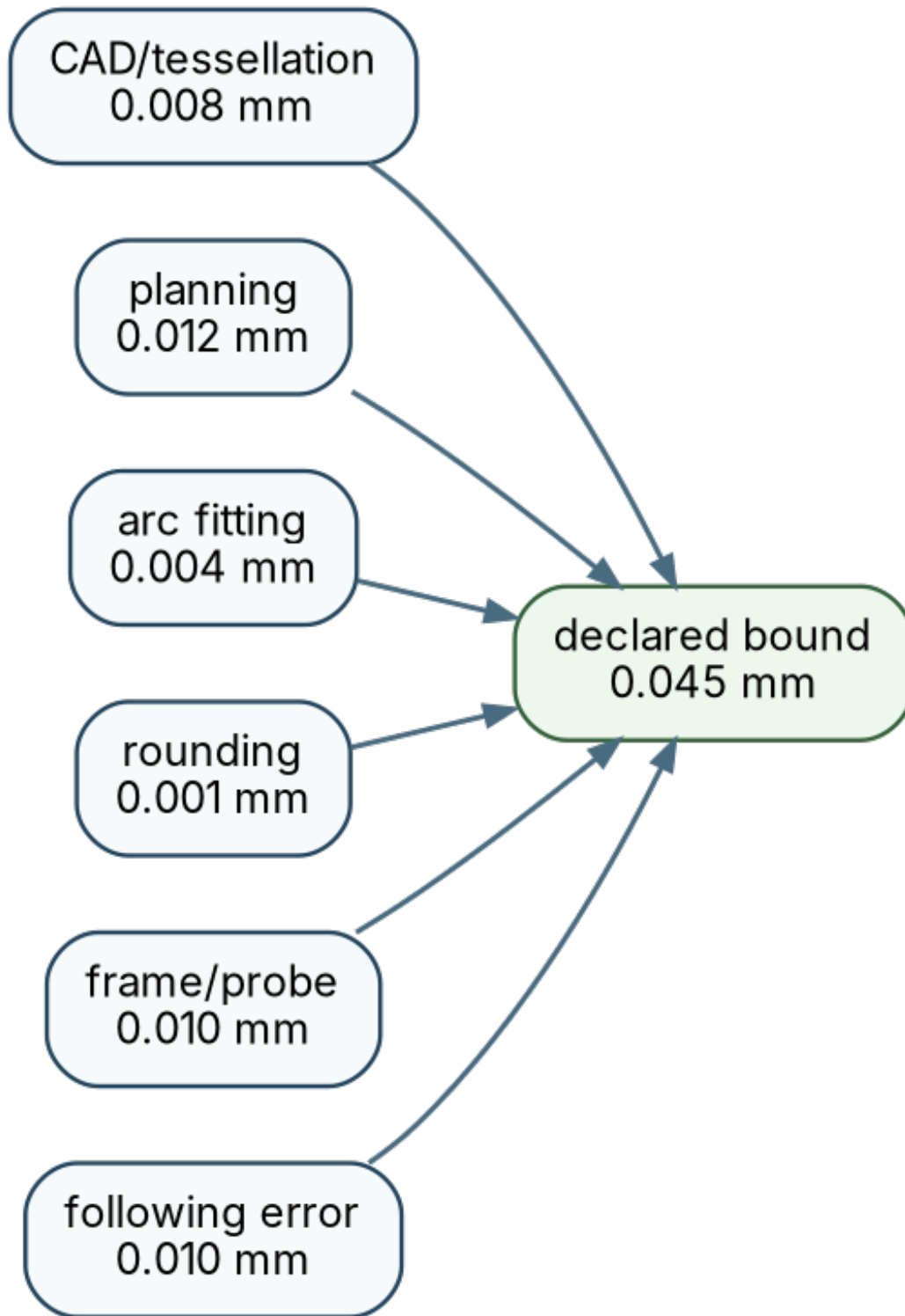


Figure 3.3: A simple additive budget for compatible worst-case position errors.

Counterexample - root-sum-square as a hard bound. Root-sum-square combination assumes a probabilistic independence model. It is not a deterministic maximum. A certificate must distinguish worst-case, probabilistic, and empirical budgets.

3.7.3 Budget-driven algorithms

A planner receives a budget slice rather than a global magic tolerance:

```
interface PlanningBudget {
  pathHausdorff: Mm;
  targetNormalError: Mm;
  topologySnap: Mm;
  maxSubdivisionDepth: number;
}
```

If the algorithm cannot meet the allocation, it refines, chooses a different representation, or fails.

3.8 Constructing a pocket path from first principles

We now build a simplified offset-contour pocket planner. The goal is pedagogy, not production geometry.

3.8.1 Restating the intent

The rectangular pocket boundary is:

$$R = [12, 42] \times [14, 34].$$

The flat end mill radius is:

$$r = 3.175/2 = 1.5875 \text{ mm.}$$

Definition - inward offset or erosion. The erosion of a region R by a tool-radius disk B_r is the set of tool-center positions whose translated disk remains inside R :

$$R \ominus B_r = \{x \mid x + B_r \subseteq R\}.$$

To keep the cutter inside the pocket, the tool-center domain for a finished wall is this inward offset:

$$R_c = R \ominus B_r.$$

For an axis-aligned rectangle:

$$R_c = [13.5875, 40.4125] \times [15.5875, 32.4125].$$

3.8.2 Step-over and coverage

Let radial step-over be $s = 1.2$ mm. Repeated inward offsets create contours until the residual core is covered.

```

input: region R, tool radius r, step-over s
C0 = inwardOffset(R, r)
paths = []
while Ck is non-empty:
    paths.append(boundary(Ck))
    Ck+1 = inwardOffset(Ck, s)
return paths

```

A real implementation must handle multiple components, vanished regions, corners, islands, and numerical degeneracy.

Definition - coverage witness. A coverage witness demonstrates that the union of guaranteed inner cutting sweeps contains the required removal region.

A checker can rasterize the required region conservatively or use exact polygonal offsets. For each cell or subregion, it verifies inclusion in at least one inner sweep.

3.8.3 Depth layers

Suppose maximum axial depth per pass is 1.5 mm. To rough to -3.8 mm, leaving 0.2 mm axial stock, choose layers:

```

-1.5 mm
-3.0 mm
-3.8 mm

```

The last increment is 0.8 mm. A layer planner should derive this list rather than overshoot and clamp after path creation.

```

function depthLayers(
    top: Mm,
    target: Mm,
    maxStep: Mm,
): readonly Mm[];

```

Required properties:

- monotone descent;
- each step magnitude $\leq$ maxStep;
- final layer equals target exactly within numeric representation;
- no duplicate layer;
- finite layer count.

3.8.4 Entry

A vertical plunge is simple but may violate tool capability or process constraints. Alternatives include:

- ramp along a line;
- helical entry;
- entry through a pre-drilled hole;
- plunge in a known empty region.

Entry is a separate planning problem with a precondition on stock and tool geometry.

Worked example. A helical entry at the center of the pocket uses a helix radius large enough for T1 and small enough to fit within the current tool-center domain. Its cutting sweep is checked against stock, target floor allowance, and wall allowance. The helix is forward-only and cannot be reversed by the scheduler.

3.8.5 Direction and climb milling

For an interior contour, climb-milling direction depends on spindle rotation and whether the tool is inside or outside the boundary. Path orientation is process semantics, not arbitrary list order.

3.8.6 Linking contours

A low link between adjacent offsets may reduce retract time. It is legal only if its sweep lies inside material already removed at that layer. Otherwise, retract to a certified clearance.

3.8.7 Pocket planner witness

```
interface PocketWitness {
  toolCenterDomain: RegionArtifact;
  offsetCorrespondence: readonly OffsetWitness[];
  layerProof: DepthLayerWitness;
  coverage: CoverageWitness;
  entries: readonly EntryWitness[];
  links: readonly LinkWitness[];
  errorBudgetUse: readonly ErrorBound[];
}
```

The planner can use a fast polygon library. The checker validates the output relations under a smaller trusted kernel.

3.9 Surface planning, distance fields, and the Eikonal equation

Motivation. Pockets are largely planar. Freeform surfaces require paths that follow changing geometry and often aim for approximately constant scallop height.

3.9.1 Cutter-location field

Definition - scalar field and distance field. A scalar field assigns one number to every point in a domain. A distance field assigns distance to a selected set or boundary; a signed-distance field also records which side of the boundary the point lies on.

For a three-axis ball-end tool, a cutter-location height field gives the lowest tool-center Z that avoids penetrating the target at each XY position. Conceptually, this is related to a morphological dilation or Minkowski construction between the target surface and reflected tool.

A sampled implementation may query nearby triangles, compute candidate contacts, and take the maximum required center height. Its output is an approximation whose error depends on:

- target representation;
- spatial index completeness;

- tool model;
- XY sampling;
- interpolation;
- numeric predicates.

3.9.2 Scallop height intuition

When parallel passes are separated, cusps of uncut material remain. For a ball of effective radius R and small step-over s on a locally flat surface, scallop height is approximately:

$$h \approx \frac{s^2}{8R}.$$

Solving for step-over:

$$s \approx \sqrt{8Rh}.$$

This approximation motivates spacing but does not by itself prove a bound on a curved surface with a finite mesh and uncertain tool.

Counterexample - naming an algorithm after the goal. A function called `constantScallop` is not evidence that physical scallop is bounded. The claim requires a relation among surface curvature, tool geometry, field discretization, path extraction, machine following, and the chosen metric.

3.9.3 Distance fields and Eikonal methods

Definition - Eikonal equation. The Eikonal equation relates an arrival-time field T to local propagation speed F :

$$|\nabla T(x)|F(x) = 1.$$

It says that arrival time changes more slowly where propagation is fast and more quickly where propagation is slow.

Definition - geodesic. A geodesic is a locally shortest path under a chosen geometry or cost metric. On a surface it generalizes a straight line; in a weighted field it avoids expensive regions.

Fast marching computes solutions for monotonically advancing fronts efficiently [R22]. Distance and arrival-time fields can support:

- geodesic distance on a surface;
- offset-front propagation;
- obstacle-aware link costs;
- contour extraction at chosen field levels;
- path planning around high-cost or forbidden regions.

A field solver is a numerical primitive, not a complete CAM guarantee. The compiler still needs:

- a precise field definition tied to manufacturing meaning;
- discretization consistency and error bounds;
- robust contour topology;

- continuous path reconstruction;
- independent checking against tool, target, and stock.

3.9.4 Hybrid strategies

Practical freeform planning often combines:

- Z-level waterlines for steep regions;
- raster or scallop-like paths for shallow regions;
- transition curves between them;
- rest machining based on remaining stock.

The intent IR should state the desired tolerance and protected regions. The strategy may choose the hybrid decomposition and return a witness for coverage and overlap.

3.10 Scheduling as an operations-research problem

Motivation. Once paths exist, their order can dominate tool changes and non-cutting time. But not every permutation is legal.

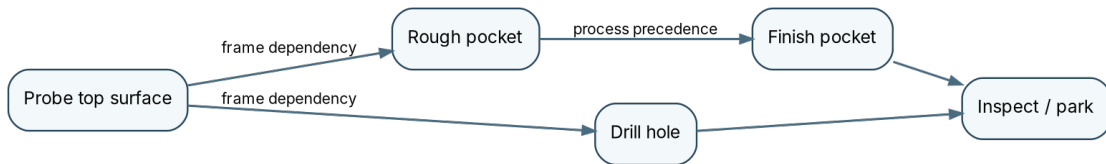


Figure 3.4: The recurring job has precedence and measurement dependencies.

3.10.1 Precedence graph

Definition - precedence graph. A precedence graph has an edge $a \rightarrow b$ when operation a must occur before b . It must be a **directed acyclic graph (DAG)**: a directed graph with no directed cycle, because a cycle would require an operation to precede itself.

Sources of precedence include:

- rough before finish;
- probe before dependent operations;
- pilot hole before larger drill;
- internal features before releasing surrounding material;
- stock-support constraints;
- inspection before a destructive setup change.

Definition - topological ordering. A topological ordering lists the vertices of a directed acyclic graph so every predecessor appears before its successor. It exists exactly when the directed graph has no cycle.

A schedule begins with such an ordering and augments it with tool, orientation, entry, and link choices.

3.10.2 Routing with precedence

If each operation has an entry and exit and transition costs, ordering resembles a traveling-salesman problem with precedence constraints. CNC toolpath sequencing has been modeled this way [R24].

A mixed-integer model can use binary variables x_{ij} indicating whether operation j follows i , plus precedence and subtour constraints. For small jobs, dynamic programming or exact search may be feasible. For larger jobs, heuristics propose candidates that an independent checker validates.

3.10.3 Worked example: two legal schedules

Assume the following estimated costs in seconds:

Action	Cost
Probe top	18
Rough pocket with T1	120
Finish pocket with T1	40
Change tool	25
Drill with T2	20
Extra long traverse when drilling first	12

Two precedence-correct schedules are:

```
A: probe -> rough(T1) -> finish(T1) -> change -> drill(T2)
B: probe -> change -> drill(T2) -> change -> rough(T1) -> finish(T1)
```

Ignoring common operation time, A pays one tool change, while B pays two. If B also incurs the long traverse, A is 37 seconds cheaper. This arithmetic does not prove A feasible: the checker must still establish that delaying drilling does not violate process or support constraints.

3.10.4 Effects and commutativity

Geometric removal is set difference, and set difference with two fixed removed regions commutes:

$$(S \setminus R_1) \setminus R_2 = (S \setminus R_2) \setminus R_1.$$

Yet machining operations may not commute because intermediate stock affects:

- entry access;
- link clearance;
- thin-wall support;
- workholding stiffness;
- probe results;
- chip evacuation;
- thermal state.

Effect summaries help a checker decide whether reordering is permitted:

```
interface OperationEffects {
  readsStock: RegionSet;
  removesStock: RegionSet;
  readsMeasurements: readonly MeasurementId[];
  writesMeasurements: readonly MeasurementId[];
```

```

requiresTool: ToolId;
requiresFrame: FrameId;
processTags: readonly string[];
}

```

Counterexample - disjoint cuts always commute. Two cuts in disjoint XY regions can still be coupled if the first removes a support tab that keeps the second region rigid.

3.10.5 Feasibility checking

The scheduler’s witness includes:

- permutation of required operations;
- chosen orientations and entries;
- predecessor satisfaction;
- tool-change sequence;
- referenced stock state for each link;
- recomputable transition costs;
- optional lower bound on optimal cost.

A checker verifies feasibility and recomputes the objective. It should call the result “optimal” only if a solver certificate or valid lower bound establishes an optimality gap.

3.11 Feed scheduling and time-optimal parameterization

Motivation. A geometric path does not determine whether the machine can follow it at the requested feed.

Let $q(s)$ be a path in joint or axis configuration and $s(t)$ its progress. Then:

$$\dot{q} = q'(s)\dot{s}$$

$$\ddot{q} = q''(s)\dot{s}^2 + q'(s)\ddot{s}.$$

Axis velocity and acceleration limits create constraints on $\dot{s}$ and $\ddot{s}$. Jerk limits introduce the third derivative. Process constraints add chip load, spindle power, engagement, and contour-error limits.

A time-optimal problem is:

$$\min T$$

subject to:

$$|\dot{q}_i| \leq v_i^{max},$$

$$|\ddot{q}_i| \leq a_i^{max},$$

$$|\ddot{q}_i| \leq j_i^{max},$$

plus process and tracking constraints. Reachability-based methods solve important forms of time-optimal path parameterization [R25].

3.11.1 Curvature constraint

For Cartesian speed v along curvature κ , normal acceleration is:

$$a_n = v^2 \kappa.$$

Thus:

$$v \leq \sqrt{\frac{a_n^{max}}{\kappa}}.$$

Sharp corners have unbounded ideal curvature. A controller may stop, blend, or deviate. The compiler needs a controller motion model or a conservative bound on blending behavior.

3.11.2 Controller versus compiler scheduling

Some controllers perform their own lookahead and acceleration planning. The CAM compiler may specify feeds but not exact $s(t)$. Then the target semantics is nondeterministic within a controller profile:

all trajectories the controller may produce
under lookahead, blend, and limit settings

Safety checks must cover that set or constrain controller settings. Assuming one nominal time law is insufficient.

Worked example. The pocket corners can be emitted as exact stops or rounded arcs. Exact stops preserve the rectangle but increase time and marks. Rounded corners improve continuity but alter the swept volume and must remain within allowance. The choice is a bounded geometric and dynamic refinement.

3.12 Combining planning and checking

The recommended architecture alternates powerful producers with narrow checkers:

```
intent
-> strategy proposes paths + witness
-> geometry checker validates target, coverage, and path bounds
-> scheduler proposes order + witness
-> schedule checker validates precedence and stock-state effects
-> linker proposes traverses + witness
-> collision checker validates full assembly sweeps
-> feed planner proposes time law or feed profile
-> dynamics checker validates machine constraints
```

This separation avoids requiring one monolithic “CAM verifier” to trust every heuristic.

3.12.1 What simulation remains good for

Definition - simulation. Simulation executes selected modeled states or traces to observe their consequences. Unless paired with a covering theorem or exhaustive finite model, it does not establish a universal property.

Simulation is still valuable for:

- visualization;
- debugging;
- regression tests;
- counterexample discovery;
- performance estimation;
- operator understanding;
- checking claims that are explicitly empirical or sampled.

The mistake is not using simulation. The mistake is promoting “no failure observed” into a universal proposition without a covering argument.

3.13 Chapter summary

Planning is best understood as constructing a feasible witness and then optimizing among feasible witnesses.

- Swept volumes state cutting and collision effects over continuous motion.
- Cutting geometry and full assembly geometry support different claims.
- Configuration space turns free-space linking into path planning around expanded obstacles.
- Stock representation determines what can be represented and proved.
- Samples require a theorem, enclosure, or adaptive bound to support continuous claims.
- Robust predicates protect topology; interval enclosures protect numerical bounds.
- Error budgets are typed and compositional, not one unexplained scalar.
- Pocket and surface strategies propose paths; independent checks establish coverage, no-gouge, and clearance.
- Scheduling combines precedence, state-dependent effects, and routing costs.
- Feed scheduling is constrained time parameterization, not a constant attached to a point list.
- Optimization never replaces feasibility checking.

3.14 Exercises

3.14.1 Swept volumes and stock

1. State separate mathematical claims for cutter no-gouge, holder collision avoidance, and guaranteed required removal.
2. A cylindrical tool of radius 2 mm translates along a 20 mm line. Describe the ideal sweep geometrically.
3. Give a case where a height field represents stock exactly and a case where it cannot.
4. Design cell classifications for a conservative voxel collision checker.
5. Explain why checking a link against final stock is unsound.
6. Draw a tool assembly that clears a pocket with its cutter but collides with its holder.

3.14.2 Continuous bounds and robustness

7. Derive the chord-count formula for a circular arc under a sagitta tolerance.
8. Compute the minimum number of chords for a 180-degree arc of radius 25 mm with 0.005 mm maximum sagitta.
9. Give a Lipschitz-based safety argument for a sampled distance function.
10. Construct a sampling counterexample with an obstacle narrower than the sample spacing.
11. Implement `orient2d` with ordinary floating point and search for nearly collinear inputs that make the sign unstable.
12. Explain why topological snapping needs its own error budget.
13. Design an interval-subdivision checker for a cubic Bezier's bounding box.

3.14.3 Error budgets

14. Classify these as numerical, model, calibration, or process uncertainty: mesh tessellation, tool runout, output rounding, thermal growth, G54 probing, arc linearization.
15. Convert an angular uncertainty of 0.0002 radians to a positional bound at radii 20, 50, and 100 mm.
16. Build a 0.04 mm worst-case budget for the pocket floor and state the metric of every term.
17. Give an example in which pass sensitivity $L > 1$.
18. Explain why a probabilistic 3-sigma bound should not be labeled a deterministic maximum.

3.14.4 Pocket planning

19. Compute the finished tool-center rectangle for the running pocket and T1.
20. Generate depth layers for top 0, target -3.8, and maximum step 1.3 mm.
21. Write pseudocode that handles multiple connected components after inward offsets.
22. Define a coverage witness for a rectangular pocket using conservative grid cells.
23. Specify the preconditions for a helical entry.
24. Explain how climb-milling direction changes for an interior versus exterior contour.
25. Design a link rule that permits low links only through definitely removed stock.

3.14.5 Surface planning

26. Use $h \approx s^2/(8R)$ to compute approximate step-over for a 3 mm ball radius and 0.01 mm scallop.
27. List the additional facts needed to turn that approximation into a certified curved-surface bound.
28. Explain the Eikonal equation in arrival-time language and give one CAM use.
29. Design a hybrid steep/shallow surface strategy and identify the transition obligations.

3.14.6 Operations research

30. Construct a precedence DAG for probe, rough, semi-finish, finish, drill, deburr, and park.
31. Find two valid topological orders and compare tool changes.
32. Give an effect-based reason two apparently independent operations do not commute.
33. Formulate binary decision variables for choosing whether operation i precedes j .
34. Explain the difference between a feasible schedule, a locally improved schedule, and a schedule with a certified optimality gap.
35. For curvature 0.2 mm^{-1} and maximum normal acceleration 500 mm/s^2 , compute the speed bound in mm/s and mm/min.

3.14.7 Project

36. Implement the simplified rectangular-pocket planner with exact axis-aligned offsets.

37. Produce a witness containing depth layers, path orientation, and a grid coverage map. Write a separate checker.
38. Implement a conservative retract-XY-descend linker and a checker using rectangular stock and fixture boxes.
39. Add a scheduler that minimizes tool changes subject to a precedence DAG. Validate its result independently.
40. Add an error-budget object and make planning fail when the allocation is exceeded.

Chapter 4

Building Trust: Invariants, Certificates, and Runtime Assurance

A planner can produce an elegant path and still be wrong. A postprocessor can preserve the path and still emit the wrong modal program. An upload can succeed while the reply is lost. A correct file can run under the wrong work offset or tool. Trust therefore cannot be a final checkbox added after compilation. It is a chain of precise claims, independent checks, content identity, and runtime assumption discharge.

By the end of this chapter, you should be able to:

- derive command preconditions with Hoare logic and weakest preconditions;
- compute conservative machine-state facts with abstract interpretation;
- design property-specific claims, evidence types, and certificate dependency graphs;
- explain proof-carrying CAM and the trusted computing base;
- validate the exact serialized controller bytes;
- model upload, start, hold, resume, abort, alarm, and timeout as a temporal state machine;
- bind a checked job to live machine, tool, frame, and setup state;
- design a staged implementation roadmap for Dropcut and the Z1.

4.1 Start by naming the proposition

Motivation. A status such as verified, safe, or checked to 0.1 mm is impossible to interpret without knowing what proposition was checked, on which artifact, by which method, and under which assumptions.

Compare:

```
{ safe: true }
```

with:

```
{  
  subject: "sha256:job-bytes...",  
  proposition: {  
    kind: "max-target-penetration",  
    target: "sha256:target...",  
    maximum: mm(0.02),  
    metric: "signed-normal-depth"  }  
}
```

```

},
result: "proved-bounded",
method: "interval-swept-volume-v2",
assumptions: ["tool-measurement-A7", "G54-bound-B2"],
evidence: ["sha256:subdivision-tree..."],
checker: "cam-geometry-checker@1.3.0"
}

```

The second object can be reviewed, invalidated, and independently checked.

Definition - claim. A claim is a precise proposition about a precise subject artifact.

Definition - evidence. Evidence is data accepted by a specified checker as support for a claim.

Definition - certificate. A certificate is a machine-checkable structure binding claims, subjects, assumptions, evidence, dependencies, and checker identities.

Definition - proof obligation. A proof obligation is a proposition that must be established before a compiler stage, policy gate, or runtime transition is accepted. A checker discharges an obligation or returns refuted/inconclusive evidence.

Definition - attestation. An attestation establishes that an identified actor or device made a statement about identified bytes or state. It does not establish the semantic truth of that statement.

A signature can show that a controller reported hash h . It does not prove that h is collision-free.

4.1.1 Separate proposition, method, and result

```

type ClaimResult =
| "proved-exact"
| "proved-bounded"
| "translation-validated"
| "exhaustive-finite-check"
| "simulation-only"
| "assumed"
| "unknown"
| "refuted";

```

The same method can produce different results. A sampled simulation may find a collision and refute a claim. Failure to find one remains simulation-only.

Counterexample - evidence laundering. A single dextral simulation result is copied into rows labeled “gouge,” “rapid,” and “travel.” If the simulator never received a target solid or machine-axis model, it cannot support those propositions regardless of the labels attached afterward.

4.2 Hoare logic turns command requirements into derivations

Motivation. Preflight checklists drift when they are maintained separately from the commands they are supposed to justify.

Chapter 1 introduced Hoare triples:

$$\{P\} c \{Q\}.$$

We now use them to derive requirements.

4.2.1 Basic command contracts

Tool selection:

$$\{\text{ControllerIdle} \wedge \text{SpindleOff} \wedge \text{ToolAvailable}(T)\} \text{SelectTool}(T) \{\text{ActiveTool} = T\}.$$

Spindle start:

$$\{\text{ActiveTool} = T \wedge rpm \in \text{AllowedRpm}(T, M)\} \text{StartSpindle}(rpm) \{\text{SpindleRunning}(rpm)\}.$$

Cut:

$$\{\text{ReadyToCut}(\sigma, \gamma, T, f)\} \text{Cut}(\gamma, T, f) \{\text{Pose} = \text{end}(\gamma) \wedge S' = S \setminus R\}.$$

The predicate `ReadyToCut` can expand into homing, frame, tool, spindle, feed, geometry, travel, and authorization obligations.

4.2.2 Weakest preconditions

Definition - weakest precondition. $wp(c, Q)$ is the least restrictive predicate that must hold before command c so that postcondition Q holds after successful execution.

For a sequence:

$$wp(c_1; c_2, Q) = wp(c_1, wp(c_2, Q)).$$

For a conditional:

$$wp(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) = (b \Rightarrow wp(c_1, Q)) \wedge (\neg b \Rightarrow wp(c_2, Q)).$$

4.2.3 Worked derivation for the pocket epilogue

Desired final condition:

$$Q = \text{SpindleOff} \wedge \text{Pose} = p_{\text{park}} \wedge \text{GoodFinal}(S).$$

Program:

```

finish pocket
retract to clearance
move to park
stop spindle

```

Working backward:

1. stop spindle requires a valid connected controller state and establishes SpindleOff.
2. move to park requires a collision-free traverse from the retract point to p_{park} and preserves stock.
3. retract requires a collision-free vertical sweep and preserves stock.
4. finish pocket requires the rough-stock precondition, T1, spindle state, frame, travel, and a path whose cut establishes GoodFinal within budget.

The compiler can collect compile-time obligations and runtime assumptions from this derivation.

4.2.4 Loop invariants

A repeated pocket-layer loop needs an invariant. One useful invariant after completing layer k is:

```

all material definitely removed above completed depth dk
outside protected allowance;
stock below dk remains conservatively unchanged;
machine is at a certified exit pose;
spindle and tool state remain valid.

```

Initialization proves it before the first layer; preservation proves one more layer; termination plus final depth proves the roughing postcondition.

Counterexample - assertions as comments. Writing `assert(pathSafe)` after path generation does not prove the predicate. An assertion is useful only when it is checked by trusted code or discharged by evidence.

4.3 Abstract interpretation checks all represented states

Motivation. A concrete simulator follows one initial state. A final G-code checker may need to cover several possible modal states, position intervals, probe outcomes, or branches.

Definition - abstract domain. An abstract domain is a set of summary values, ordered by precision, used to represent sets of concrete states. Intervals, position boxes, and finite sets of modal states are examples.

Definition - abstract interpretation. Abstract interpretation executes a program over an abstract domain whose elements conservatively represent sets of concrete states [R6].

Fundamentals aside - orders, lattices, and fixed points. Abstract values are ordered by precision: one value may represent a subset of the states represented by another. A **join** combines alternatives conservatively. A **lattice** is an ordered structure in which joins and meets exist. A **fixed point** is a value unchanged by an analysis iteration; loop invariants are commonly computed as fixed points. A **widening** forces convergence by moving to a less precise abstract value after repeated growth.

A familiar toy example is the “sign” abstraction. Instead of evaluating every integer, use {negative, zero, positive, unknown}. Multiplying negative by positive produces negative; adding positive and negative may produce unknown. The result is less precise than concrete execution but covers many values at once. Machine-state analysis applies the same idea to positions, modes, tools, and alarms.

Let C be concrete states and A abstract states. A concretization function:

$$\gamma : A \rightarrow \mathcal{P}(C)$$

maps each abstract value to the concrete states it represents. An abstract transfer $\widehat{F}$ is sound when:

$$F(\gamma(a)) \subseteq \gamma(\widehat{F}(a)).$$

4.3.1 A small abstract machine domain

```
interface AbstractMachineState {
  position: Box3 | "unknown";
  homing: "homed" | "unhomed" | "maybe";
  tool: ToolRef | ReadonlySet<ToolRef> | "unknown";
  spindle: "off" | Interval<Rpm> | "unknown";
  wcs: TransformInterval | "unknown";
  distanceMode: ReadonlySet<"absolute" | "incremental">;
  units: ReadonlySet<"mm" | "inch">;
  motionMode: ReadonlySet<"rapid" | "linear" | "cwArc" | "ccwArc" | "probe">;
  alarm: "yes" | "no" | "maybe";
}
```

The domain should be driven by claims. If the only claim is travel bounds, exact coolant state may be irrelevant. If the claim is safe spindle shutdown, it matters.

4.3.2 Transfer functions

For G90, distance mode becomes {absolute}. For G91, it becomes {incremental}. For an absolute move to X=10 with rounding interval ρ :

$$X' = [10 - \rho, 10 + \rho].$$

For an incremental move by 10:

$$X' = X + [10 - \rho, 10 + \rho].$$

If the incoming mode is {absolute, incremental}, the successor joins both possibilities. The result may become too imprecise to prove travel safety, which is the correct conservative outcome.

4.3.3 Join and fixed point

At a control-flow merge, abstract states join: the result represents every state from every incoming branch. If one branch has T1 and another T2, the merged tool state is {T1, T2}. A later cut requiring definitely T1 cannot be proved unless a guard or tool selection re-establishes it.

Loops require fixed-point iteration because information flows around the loop until it stops changing. A widening operator may accelerate convergence by deliberately sacrificing precision. Production controller dialects can also forbid unbounded macros and loops, reducing the analysis problem.

4.3.4 Proof-carrying abstract states

An untrusted analyzer can emit an invariant before and after every block:

```
interface BlockInvariant {
  block: number;
  before: AbstractMachineState;
  after: AbstractMachineState;
}
```

A small checker verifies:

1. the initial abstract state covers all allowed initial states;
2. each abstract transfer covers the block's concrete semantics;
3. adjacent states connect;
4. each local safety predicate follows;
5. the final state satisfies policy.

Worked example. The checker proves that every cutting block has definite mm, absolute, T1 or T2 as required, spindle on, known G54, and a position box inside machine travel. It proves the final spindle state is off. A raw unknown block resets affected components to unknown, causing the production policy to reject unless later commands re-establish them.

Counterexample - optimistic unknown. Treating unknown as “probably unchanged” makes the analysis unsound. Unknown must represent every possible value in the affected domain.

4.4 Translation validation checks each actual pass result

Motivation. Full formal verification of a geometry-heavy TypeScript compiler is expensive. We still want confidence in actual outputs.

Translation validation inserts a checker after a transformation [R14]. The transformer may be complex and untrusted. The checker validates the relation for the produced input-output pair.

4.4.1 Arc linearization checker

Inputs:

- source arc with start, end, center, axis, sweep;
- output line segments;
- error budget ϵ .

Checker obligations:

1. output begins at source start and ends at source end within endpoint bounds;
2. vertex order follows the declared sweep;
3. every chord endpoint lies on or near the source arc under a bound;

4. maximum sagitta is at most ε ;
5. helical axial interpolation is preserved;
6. provenance and error-budget use are complete.

4.4.2 Path-reordering checker

Inputs:

- original set of operations and dependency graph;
- proposed order and orientations;
- effect summaries;
- transition witness.

Checker obligations:

- every required operation appears exactly once;
- no unknown operation appears;
- all precedence edges are respected;
- reversed paths are reversible;
- every link is checked at the correct stock state;
- final process effect satisfies the same intent;
- reported cost is recomputed.

4.4.3 Modal-compression checker

The checker interprets explicit Controller IR and compressed bytes from the same initial modal state. It compares canonical event traces and final modal state, allowing only declared numeric bounds.

4.4.4 Why checkers can be smaller

A planner must discover a path among many possibilities. A checker receives the proposed path and verifies local conditions. Search is often harder than checking. This asymmetry is the foundation of certificate-carrying architecture.

Definition - checker independence. Checker independence is the degree to which a checker avoids the producer’s code paths, representations, hidden state, and likely defect modes. Independence is strengthened by separate implementations, simpler algorithms, different data representations, and restricted schemas.

Counterexample - shared normalization bug. If both postprocessor and checker call the same helper that mistakenly interprets arc-center offsets, the “independent” check repeats the defect.

4.5 The certificate graph

Motivation. One final certificate cannot be a flat table because claims depend on different artifacts and assumptions at different stages.

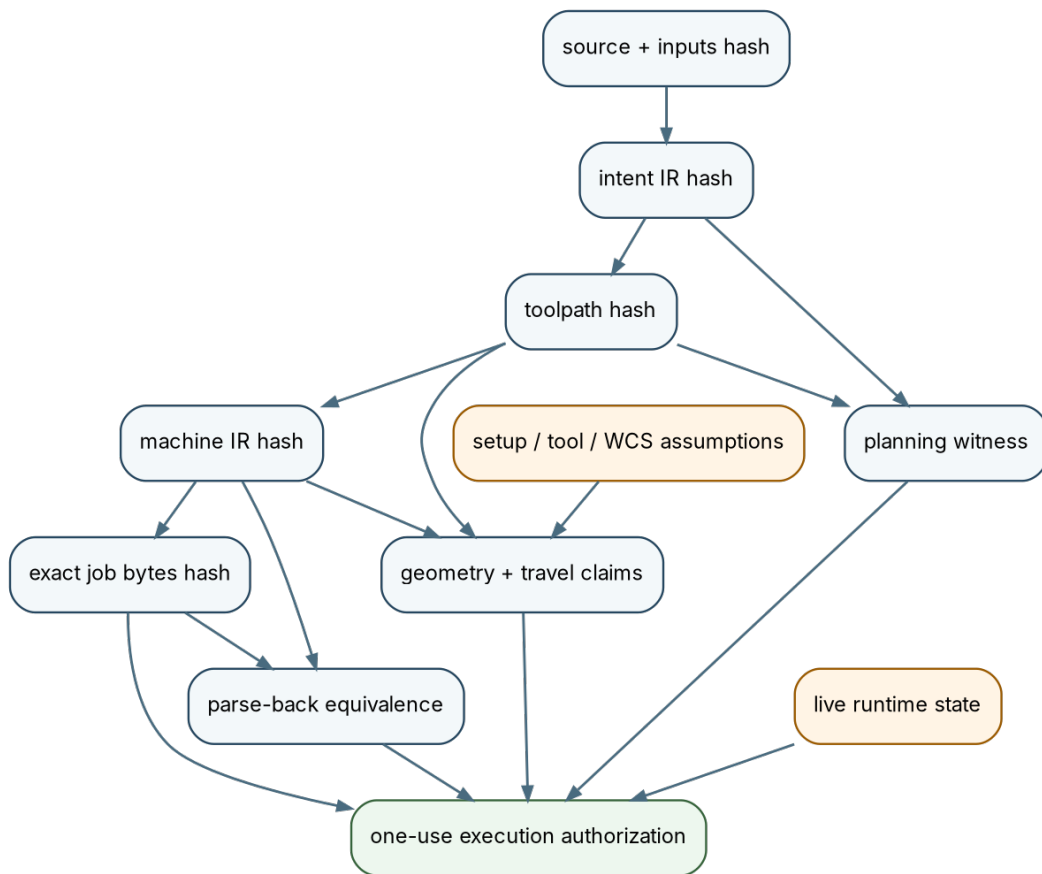


Figure 4.1: A certificate graph binds claims to exact artifacts and runtime state.

Definition - directed acyclic graph (DAG). A DAG is a directed graph with no directed cycle. Certificate dependencies form a DAG so claims can be checked in dependency order and cannot justify themselves through a cycle.

4.5.1 Schema

```
interface Claim {
  id: ClaimId;
  subject: ArtifactRef;
  proposition: StructuredPredicate;
  result: ClaimResult;
  method: MethodRef;
  assumptions: readonly AssumptionRef[];
  evidence: readonly EvidenceRef[];
  dependencies: readonly ClaimId[];
  bound?: ErrorBound;
  checker: {
    id: string;
    version: string;
    binaryHash: Hash;
    relationVersion: string;
  };
}
```

Example propositions:

```
type StructuredPredicate =
  | {
    kind: "travel-contained";
    trajectory: Hash;
    machineEnvelope: Hash;
  }
  | {
    kind: "disjoint";
    leftSweep: Hash;
    rightObstacle: Hash;
    minimumSeparation?: Mm;
  }
  | {
    kind: "max-target-penetration";
    sweep: Hash;
    target: Hash;
    maximum: Mm;
    metric: "normal-depth" | "euclidean-depth";
  }
  | {
    kind: "trace-refines";
    source: Hash;
    target: Hash;
    relation: RelationId;
  }
  | {
    kind: "stored-hash-equals";
    expected: Hash;
    observed: Hash;
  };
```

4.5.2 Assumptions

```
interface Assumption {
  id: AssumptionId;
  proposition: StructuredPredicate;
  source: "operator" | "calibration" | "machine" | "library";
  evidence?: ArtifactRef;
  runtimeCheck?: RuntimeCheckSpec;
  validUntil?: Timestamp;
}
```

Examples:

- T1 radius interval;
- fixture model identity;
- G54 transform interval;
- firmware semantics profile;
- machine following-error envelope;
- stock dimensions.

4.5.3 Evidence types

```
type Evidence =
| AbstractInterpretationTrace
| IntervalSubdivisionTree
| CollisionSeparationWitness
| CoverageWitness
| ParseBackTraceComparison
| SolverCertificate
| ModelCheckingTrace
| Counterexample
| CalibrationRecord
| SignatureAttestation;
```

An evidence type should be claim-specific enough that it cannot be accidentally applied to an unrelated proposition.

4.5.4 Dependency and invalidation

Changing any of these should invalidate dependent claims:

- source or project parameters;
- target, stock, or fixture geometry;
- tool or holder;
- frame transform;
- machine or firmware profile;
- planner or pass configuration;
- postprocessor precision;
- exact bytes.

Content hashes make invalidation mechanical.

4.5.5 Completeness policies

Different operating modes require different claims.

Policy	Minimum examples
Preview	schema, finite values, parse success
Attended air cut	travel, modal parse-back, exact bytes, runtime identity
Attended material cut	plus tool, WCS, target, fixture, holder, stock, final-state claims
Unattended production	plus protocol liveness, monitored stop behavior, calibrated dynamics, recovery policy

Definition - assurance policy. An assurance policy names the claims and result strengths required for one operating mode. It converts a large certificate graph into an explicit admission rule.

Definition - policy completeness. A certificate graph is complete for policy P when every proposition required by P is present with an acceptable result and all dependencies and assumptions are valid.

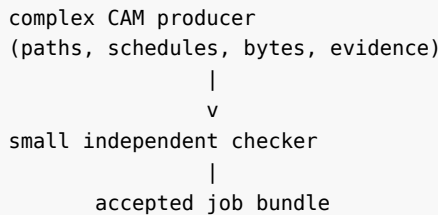
Absence of an error is not completeness.

4.6 Proof-carrying CAM and the trusted computing base

Motivation. The planner, optimizer, UI, simulator, geometry kernel, and postprocessor are too large and change too frequently to trust as one indivisible safety mechanism.

Definition - proof-carrying CAM. Proof-carrying CAM is an architecture in which a potentially complex, untrusted producer emits a job artifact plus property-specific evidence, and a smaller consumer-side checker accepts the artifact only when the evidence satisfies a declared policy.

Proof-carrying code asks an untrusted producer to supply code plus evidence that a consumer-side checker validates against a safety policy [R15, R16]. CAM can adopt the same architecture:



Definition - trusted computing base (TCB). The TCB is the hardware, software, definitions, key material, and assumptions whose correctness is necessary for the assurance result.

A deliberately small CAM TCB includes:

- definitions of claim predicates and pass relations;
- canonical hashing and serialization;
- independent Controller IR and G-code interpreters;
- robust numeric primitives used by checkers;

- certificate graph validation;
- runtime identity and hash handshake;
- a small authorization/stop monitor;
- explicitly accepted physical assumptions.

It should ideally exclude:

- UI and viewport;
- heuristic toolpath strategies;
- operation optimizer;
- main compiler orchestration;
- JavaScript runtime;
- preview simulator;
- caches.

4.6.1 Proof object or recomputation?

A checker may:

1. recompute the property independently;
2. validate a compact witness;
3. check a formal proof term;
4. combine methods.

For modal trace equivalence, recomputation is simple. For spatial coverage, a cell decomposition witness can avoid repeating planning. For a solver schedule, a dual bound or solver certificate may support optimality.

4.6.2 Checker hardening

Evidence can be malformed or adversarial. Checkers need:

- strict schemas and version rules;
- size and recursion limits;
- deterministic resource bounds;
- integer-overflow and NaN defenses;
- no execution of evidence-supplied code;
- explicit rejection of unknown claim/evidence kinds;
- reproducible diagnostics.

Counterexample - signed but unchecked evidence. A signed certificate from the same buggy producer authenticates the bug. Producer identity is useful, but semantic checking remains necessary.

4.7 The exact bytes are the deployed artifact

Motivation. A correct Machine IR can still be corrupted by serialization, line endings, encoding, upload truncation, or controller interpretation.

Definition - controller semantics profile. A controller semantics profile is a versioned artifact defining the target dialect's parsing, modal groups, numeric conventions, motion interpretation, supported extensions, and relevant lifecycle behavior for a firmware range.

The final-byte validation defined in Chapter 2 now becomes the first link in deployment assurance. The validation chain should be:

```
Machine IR
-> Controller IR
-> serialize exact bytes B
-> independently parse B
-> interpret under declared controller semantics
-> compare trace to Controller IR
-> hash B
-> upload B
-> obtain stored hash B'
-> require B' = B before start
```

4.7.1 Parse-back relation

The comparison may require bounded refinement rather than exact equality because coordinates are rounded. It should compare:

- event kind and order;
- coordinate frames and modes;
- path endpoints and interpolation semantics;
- tool, spindle, and accessory state;
- probe-result bindings;
- final modal and machine state;
- numeric deviation under the formatting budget.

4.7.2 Transport completeness

A transport write may write fewer bytes than requested. The sender must loop until all bytes are transferred or transition to an explicit failure/ambiguous state. A successful return from one call is not necessarily full transfer.

4.7.3 Controller storage identity

A controller may store a filename without a hash. A safer host protocol can read back content, request a controller-side digest, or wrap the file with an application-level manifest. The strength of the claim depends on what the controller can attest.

Counterexample - previewing one artifact and running another. If the viewport renders canonical commands but export applies additional arc fitting, rounding, or preamble insertion, the operator may approve a different path from the executed one. Preview data should be generated from, or cryptographically bound to, the final validated artifact.

4.8 The controller is a concurrent state machine

Motivation. `upload()`, `start()`, and `abort()` are not isolated function calls. They participate in a protocol with timeouts, acknowledgements, machine state, and possible faults.

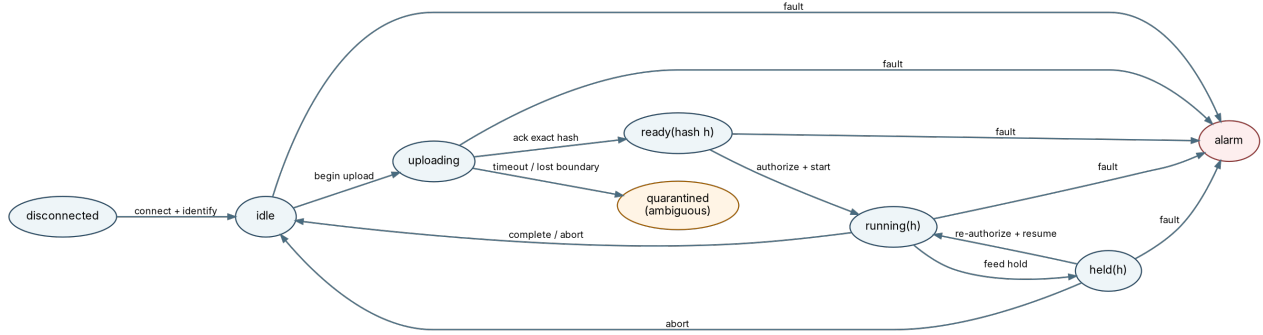


Figure 4.2: A simplified job and controller state machine.

Definition - transition system. A transition system consists of states, an initial-state predicate, and a next-state relation. A trace is a sequence connected by the relation.

Useful controller states include:

- disconnected;
- connected but unidentified;
- idle;
- uploading;
- ready with stored hash h ;
- running execution instance h ;
- held;
- alarmed;
- quarantined after ambiguity.

4.8.1 Safety invariants

$$\Box(\text{Running}(h) \Rightarrow \text{StoredHash} = h)$$

$$\Box(\text{Running}(h) \Rightarrow \text{Authorized}(h, e))$$

$$\Box(\text{Alarm} \Rightarrow \neg \text{StartAllowed})$$

$$\Box(\text{ReadOnlyCommand} \Rightarrow \neg \text{MotionEffect})$$

The last invariant requires a complete command grammar and effect classification.

4.8.2 Liveness properties

$$\Box(\text{UploadStarted} \Rightarrow \Diamond(\text{Ready} \vee \text{Failed} \vee \text{Quarantined}))$$

$$\Box(\text{AbortRequested} \Rightarrow \Diamond(\text{Stopped} \vee \text{Alarm}))$$

These statements require assumptions about controller scheduling, communication, and hardware response.

4.8.3 Atomic admission

A preflight that reads state, returns, and later sends motion has a time-of-check/time-of-use race. Another command can change tool, WCS, job, or alarm state between check and send.

A safer sequence is:

```
acquire session authority
refresh live state and state epoch e
check command/job preconditions
issue authorization bound to artifact hash and e
send/consume authorization atomically with start
observe durable acknowledgement
release or transition to monitored execution
```

Definition - state epoch. A state epoch is a monotonically changing identifier used to invalidate an authorization when relevant state changes.

4.8.4 Ambiguous timeout

If the host sends start and the reply is lost, the machine may be running or idle. Blind retry can start a second action or corrupt protocol framing.

Definition - ambiguous state. An ambiguous state conservatively represents several possible remote states after an operation with unknown outcome.

The session enters quarantine and re-synchronizes through a trusted status boundary before further state-changing commands.

Counterexample - timeout means failure. Network timeout proves only that the host did not receive a timely response. It does not prove the controller did not receive or execute the request.

4.8.5 Stop-class commands

Feed hold, abort, and emergency actions should not be blocked by the same admission lock used for ordinary motion. Their semantics and priority are distinct. However, “stop request sent” is not “motion stopped.” Completion requires observed stopped or faulted state.

Definition - model checking. Model checking explores every state reachable in a finite or finitely abstracted transition system to establish a temporal property or return a counterexample trace. It is especially useful for protocol races, stale authorizations, and ambiguous timeouts.

4.9 Runtime assurance binds proof to reality

Motivation. Compile-time claims are conditional on tool, fixture, frame, machine, and controller assumptions. Execution must check those assumptions on the live system.

4.9.1 Assumption manifest

A job bundle may require:

```
interface RuntimeAssumptionManifest {
  machineIdentity: MachineIdentityPredicate;
  firmwareProfile: FirmwareCompatibilityPredicate;
  homing: "required";
  activeWcs: TransformIntervalPredicate;
  tool: ToolIdentityAndMeasurementPredicate;
  setup: SetupIdentityPredicate;
  alarmState: "clear";
  storedJobHash: Hash;
  calibrationRecords: readonly CalibrationRequirement[];
}
```

The preflight returns evidence or a failed assumption, not a generic boolean.

4.9.2 One-use authorization

```
authorization = sign_or_mac(
  jobHash,
  machineIdentity,
  stateEpoch,
  assumptionSnapshotHash,
  expiry,
  nonce
)
```

The controller or trusted host-side monitor consumes it once. A state change or expiration invalidates it.

4.9.3 Runtime assurance monitor

Definition - runtime assurance. Runtime assurance uses a comparatively small trusted component to monitor an advanced component and intervene before a safety property can be violated.

Runtime-assurance frameworks formalize this separation between an advanced component and a smaller trusted safety mechanism [R27]. A Z1-oriented monitor can enforce:

- allowed controller-state transitions;
- no start or resume without current authorization;
- exact job-hash binding;
- command class and framing;
- feed and coordinate envelopes available at the protocol level;
- watchdog and communication-loss policy;
- durable alarm and completion events;
- stop confirmation.

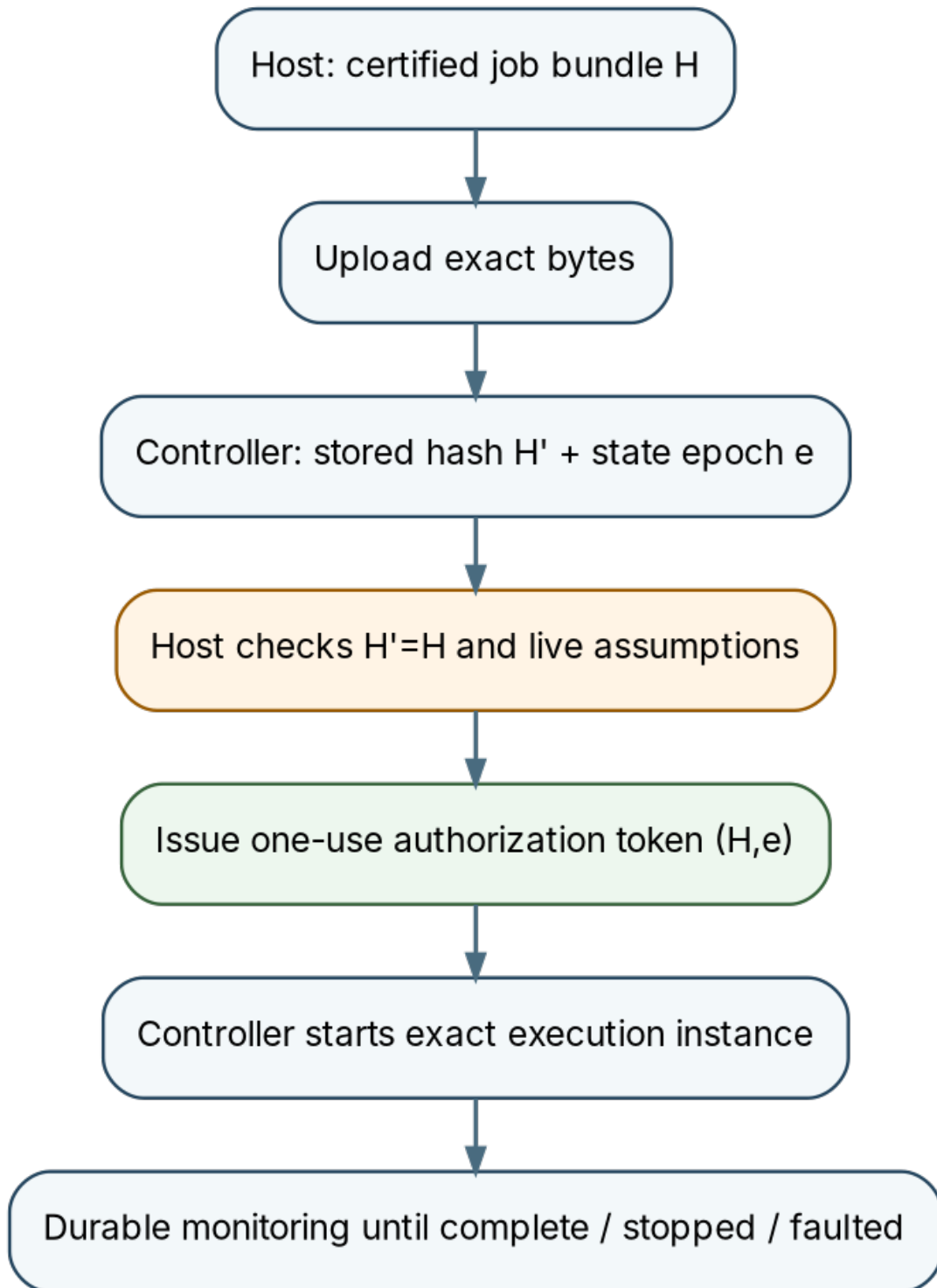


Figure 4.3: A hash-bound runtime handshake.

It cannot directly prove:

- fixture placement matches CAD;
- tool diameter is nominal without measurement;
- stock is clamped correctly;
- material behaves as modeled;
- a spindle or stepper hardware fault will not occur.

4.9.4 Stopping distance

Definition - hybrid system. A hybrid system combines discrete modes, such as running or held, with continuous dynamics, such as position and velocity. Controller authorization is discrete; braking distance evolves continuously [R26].

A stop is dynamic. Under speed v and guaranteed deceleration a :

$$d_{stop} \geq \frac{v^2}{2a}.$$

Communication and command latency Δt add at least:

$$d_{latency} \geq v\Delta t$$

before deceleration begins. Safe envelopes and monitors need these bounds, not instantaneous-stop assumptions.

Worked example. At 50 mm/s with guaranteed deceleration 500 mm/s², ideal braking distance is 2.5 mm. With 60 ms worst-case latency, add 3 mm, for at least 5.5 mm before other uncertainty. A monitor that intervenes only 2 mm before a forbidden region is too late.

4.10 Counterexamples that teach the architecture

This section gathers plausible shortcuts and identifies the missing concept.

4.10.1 “The type says validated”

Failure: a branded type does not identify claims, evidence, artifact hash, or later invalidating passes.

Repair: property-specific certificate claims attached to content-addressed artifacts.

4.10.2 “The simulator found no gouge”

Failure: target geometry may be absent; sampling may miss continuous penetration.

Repair: target-specific proposition plus conservative continuous-domain checker, or honest simulation-only status.

4.10.3 “The command starts with status”

Failure: the payload may contain additional lines or delimiters with motion effects.

Repair: closed grammar, complete parse, effect classification of every command, reject unknown syntax.

4.10.4 “Preflight passed a moment ago”

Failure: live state changed before execution.

Repair: state epoch and atomic authorization/start sequence.

4.10.5 “Upload timed out, so retry”

Failure: the first upload or start may have succeeded.

Repair: quarantine ambiguous state, re-synchronize, compare exact stored hash and lifecycle state.

4.10.6 “The preview looks right”

Failure: preview may render pre-postprocessor IR while execution uses rounded, compressed, or otherwise modified bytes.

Repair: render from final parsed bytes or bind preview and execution artifacts through checked refinement and hashes.

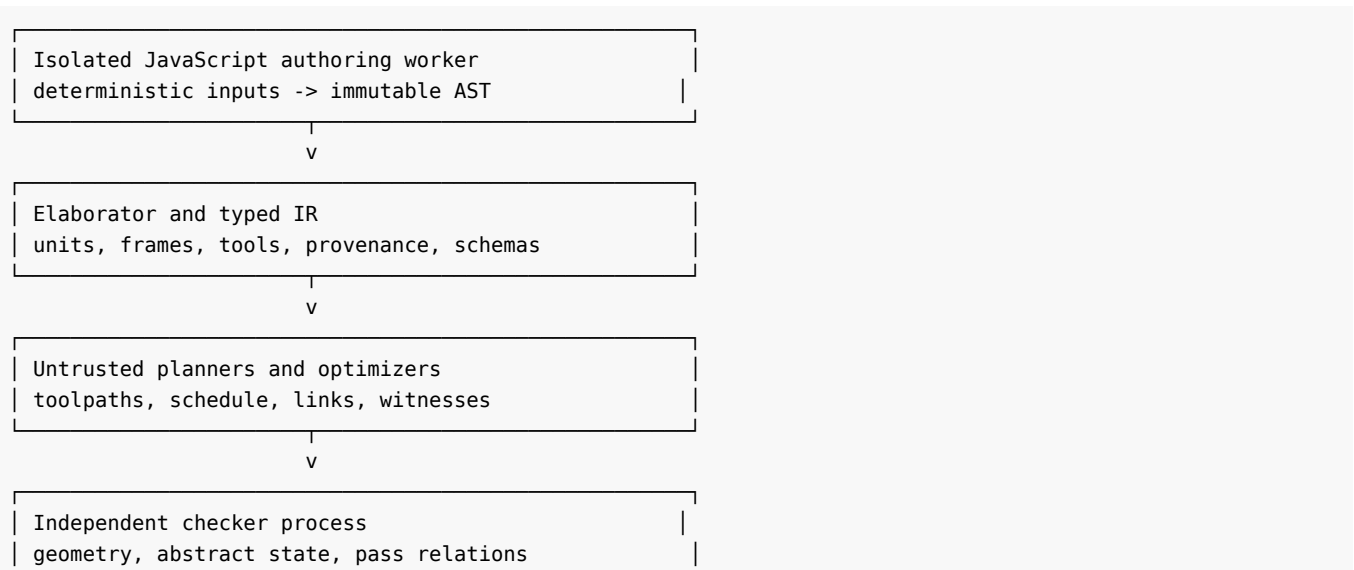
4.10.7 “The machine stopped because abort returned”

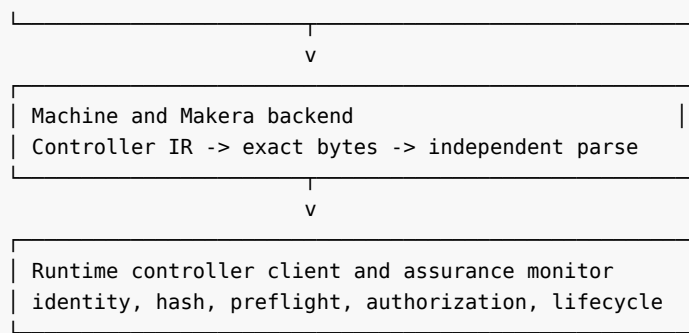
Failure: host API return may acknowledge request submission, not physical cessation.

Repair: temporal postcondition requiring observed stopped/faulted state and accounting for stopping distance.

4.11 A practical architecture for Dropcut and the Z1

A credible first implementation does not need full theorem proving. It needs honest boundaries and small checkers.





4.11.1 Suggested packages

```

@cam/semantics
@cam/ir-schema
@cam/certificate-schema
@cam/checker-core
@cam/robust-geometry
@cam/controller-semantics
@cam/runtime-assurance

```

Existing strategy, planner, and postprocessor packages can remain producers.

4.11.2 Trust boundaries

Run the compiler and checker as distinct processes when possible. The compiler emits a bundle to a checker through canonical serialization. The checker does not import planner code or accept executable callbacks. The runtime client accepts only checker-approved bundles and re-checks content hashes.

4.11.3 Capability policies

A machine profile should separate:

- compile-time supported operations;
- runtime controller capabilities;
- assumed firmware semantics;
- machine limits and calibrated dynamics;
- operations prohibited by policy even if firmware supports them.

“Controller understands command” is weaker than “project policy authorizes command.”

4.12 Staged implementation roadmap

4.12.1 Stage 0: Establish artifact identity

- canonical JSON and byte hashing;
- immutable IR objects;
- provenance and pass manifests;
- exact job-byte hash in UI, CLI, and controller client.

Acceptance test: changing one relevant input changes the appropriate downstream hash; unchanged deterministic builds reproduce hashes.

4.12.2 Stage 1: Define reference semantics

- pure canonical-command interpreter;
- Controller IR interpreter;
- restricted Makera/RS274 parser and modal interpreter;
- explicit initial and final state.

Acceptance test: source Controller IR and parsed emitted bytes produce equivalent traces on a corpus of examples and generated tests.

4.12.3 Stage 2: Add translation validators

Start with discrete or analytically bounded passes:

- traverse expansion;
- arc linearization;
- coordinate rounding;
- feed clamping;
- modal compression;
- preamble/epilogue insertion.

Acceptance test: mutations of output are rejected with localized counterexamples.

4.12.4 Stage 3: Build abstract machine analysis

- modal state;
- tool/spindle state;
- frames and position boxes;
- travel envelope;
- final shutdown policy;
- raw-command invalidation.

Acceptance test: all cutting blocks have definite required state; unknown effects fail closed.

4.12.5 Stage 4: Harden geometry checking

- robust topology predicates;
- tool and holder solids;
- target and fixture artifacts;
- conservative line/arc sweeps;
- adaptive subdivision for general curves;
- separate no-gouge, collision, and removal claims;
- typed error budgets.

Acceptance test: thin-obstacle, tangent-contact, frame-uncertainty, and near-degenerate regression cases produce proved, refuted, or inconclusive results without false proof.

4.12.6 Stage 5: Certificate graph and policies

- structured propositions;

- evidence codecs;
- checker identities;
- dependency DAG;
- policy completeness;
- human-readable assurance report generated from checked graph.

Acceptance test: no producer can mint a required claim without appropriate evidence and checker acceptance.

4.12.7 Stage 6: Controller state machine and runtime binding

- closed command grammar;
- serialized command admission;
- state epochs;
- upload/read-back or hash protocol;
- one-use start/resume authorization;
- durable safety events;
- quarantine and recovery after ambiguity;
- stop confirmation.

Acceptance test: model checking and integration fault injection find no unauthorized start, blind retry, dropped critical event, or stale-preflight execution in the modeled state space.

4.12.8 Stage 7: Optimization with checked feasibility

- precedence-aware operation scheduler;
- stock-aware link planner;
- feed/time parameterizer;
- objective recomputation and optional lower-bound certificates.

Acceptance test: optimizer can be replaced, randomized, or intentionally corrupted without bypassing feasibility checkers.

4.13 Capstone: the complete recurring job argument

We can now state the assurance case for the pocket job as a chain.

4.13.1 Source and elaboration

Claim A:

The JavaScript authoring run deterministically produced Plan IR P from source hash S and declared input hashes I.

Evidence:

- isolated evaluation record;
- AST schema validation;
- content hashes;
- elaboration diagnostics.

4.13.2 Intent and planning

Claim B:

Toolpath TP implements pocket and drill intent INT within the allocated planning metrics.

Evidence:

- offset and layer witness;
- target no-gouge check;
- coverage witness;
- entry and link checks;
- tool compatibility.

4.13.3 Scheduling and machine lowering

Claim C:

Scheduled program SP contains every required operation, respects precedence, and is feasible for Z1 profile M.

Evidence:

- permutation and DAG check;
- state-token verification;
- travel and capability analysis;
- stock-state references;
- feed limits.

4.13.4 Controller lowering

Claim D:

Exact byte artifact B boundedly refines Controller IR C under Makera semantics profile F.

Evidence:

- independent parse-back;
- modal trace comparison;
- rounding budget;
- preamble and epilogue checks;
- byte hash.

4.13.5 Runtime

Claim E:

Live execution instance E runs exact bytes B on a machine/setup satisfying assumptions A.

Evidence:

- machine and firmware identity;
- homing and alarm state;

- tool measurement and setup identity;
- G54 transform bound;
- stored hash acknowledgement;
- one-use authorization bound to state epoch;
- durable terminal status.

4.13.6 Composed conclusion

If all checkers accept and runtime assumptions hold:

Every permitted execution of E:
remains inside the machine and fixture safety envelope;
uses the required tool and process state for each cut;
executes bytes corresponding to the approved controller program;
leaves final stock within the stated target tolerance;
ends in an explicitly stopped, spindle-off state,
subject to the named physical uncertainty bounds.

The conclusion is intentionally conditional and quantitative. That is a strength. It states exactly where software certainty ends and physical responsibility begins.

4.14 Chapter summary

Trustworthy CAM compilation is a chain of explicit propositions.

- Hoare contracts and weakest preconditions derive command requirements.
- Invariants express properties preserved across passes and execution.
- Abstract interpretation covers sets of machine and modal states conservatively.
- Translation validation checks actual outputs without trusting complex transformers.
- Claims identify a subject, proposition, method, result, assumptions, evidence, and checker.
- Certificates form a dependency DAG whose hashes make invalidation mechanical.
- Proof-carrying CAM keeps the planner outside a small trusted checker core.
- Final validation covers exact serialized bytes and controller semantics.
- The controller is a concurrent transition system with safety, liveness, ambiguity, and recovery.
- Runtime assurance binds compile-time evidence to live machine, tool, frame, setup, and job identity.
- The system must fail closed or report inconclusive when a required proposition cannot be established.

4.15 Exercises

4.15.1 Contracts and invariants

1. Write Hoare triples for tool change, spindle start, probe, cut, traverse, feed hold, resume, and abort.
2. Derive the weakest precondition for `select T1; start spindle; cut path; stop spindle` under final condition `spindle off` and stock satisfies roughing postcondition.
3. State initialization and preservation obligations for the invariant “cutting implies spindle on.”
4. Design a loop invariant for peck drilling.
5. Distinguish an assertion, precondition, invariant, assumption, guarantee, witness, and attestation using one sentence each.

4.15.2 Abstract interpretation

6. Define concretization for an interval and an axis-aligned position box.

7. Write abstract transfer rules for G20, G21, G90, G91, G0, and G1.
8. Analyze a program that branches between T1 and T2 and later attempts a T1 cut.
9. Show how an unknown raw block affects modal and position state.
10. Design a widening for repeated incremental X moves.
11. Implement a block-invariant checker for a restricted G-code subset.

4.15.3 Translation validation

12. Specify a checker for coordinate rounding.
13. Specify a checker for inserting a safety retract.
14. Give a case where two path orders have the same final stock but the reorder checker must reject.
15. Explain how independent implementations can still share a modeling assumption and fail together.
16. Create mutation tests for a modal-compression checker.

4.15.4 Certificates

17. Write structured claims for travel, holder collision, target gouge, required removal, and final spindle state.
18. Design separate evidence schemas for target no-gouge and rapid-through-stock.
19. Draw the certificate DAG for the running job and mark which nodes change after replacing T1.
20. Define completeness policies for preview, air cut, attended cut, and unattended cut.
21. Explain why a signature over a certificate graph is useful but insufficient.
22. Design resource limits for an untrusted evidence checker.

4.15.5 Controller protocol

23. Define states and transitions for connect, identify, upload, ready, start, hold, resume, complete, abort, alarm, disconnect, and quarantine.
24. State four safety invariants and two liveness properties.
25. Construct a time-of-check/time-of-use race involving G54.
26. Model a lost start acknowledgement as a set of possible states.
27. Explain why stop-class commands require a different admission path.
28. Design a closed grammar for read-only controller requests.
29. Decide which events may use best-effort telemetry and which require durable delivery.

4.15.6 Runtime assurance

30. Design a runtime assumption manifest for the pocket job.
31. At 40 mm/s, 400 mm/s² deceleration, and 80 ms latency, compute a lower bound on stopping distance.
32. List five facts a software monitor can enforce and five physical facts it can only assume or measure indirectly.
33. Design a one-use authorization token and state the replay protections.
34. Specify quarantine recovery after an ambiguous upload.
35. Explain how a changed G54 can invalidate an unchanged job hash.

4.15.7 Capstone implementation

36. Implement a certificate graph schema and topological dependency checker.
37. Extend the Chapter 2 mini-controller interpreter with abstract state and final-state policy.
38. Implement exact-byte serialization, parse-back, hashing, and a fake controller that reports stored hashes.

39. Implement a state-epoch preflight/start handshake and write concurrency tests that attempt stale authorization.
40. Produce a machine-readable and human-readable assurance report for the recurring job. Each human statement must be traceable to a checked claim ID.

Glossary

Abstract domain. A mathematical domain whose values conservatively represent sets of concrete values or states. Intervals, position boxes, sets of modal states, and tool sets are examples.

Abstract interpretation. Systematic execution of a program over an abstract domain to compute sound facts about all represented executions.

Abstraction function. A mapping that forgets lower-level detail while preserving observations relevant at a higher level, such as mapping controller events to material-removal outcomes.

Admissible configuration. A machine pose or joint state satisfying travel, collision, process, and policy constraints.

Artifact. An immutable compiler input or output such as source, IR, geometry, evidence, machine profile, or exact controller bytes.

Assertion. A proposition intended to hold at a specific point or for a specific artifact.

Assumption. A fact required by a claim but not established by the claim's checker.

Attestation. Authenticated evidence that an actor or device made a statement about identified data or state.

Authoring language. The convenient language used by a person to construct a job, including macros, loops, and libraries.

Bounded refinement. A refinement that permits quantified deviation under a named metric.

Canonical command. A machine-independent action with explicit physical meaning, such as cut, traverse, probe, or spindle start.

Certificate. A machine-checkable graph binding artifacts, claims, assumptions, evidence, dependencies, and checker identities.

Claim. A precise proposition about a precise artifact.

Clearance. A geometric separation requirement between the moving assembly and stock, target, fixtures, or machine structures.

Compiler pass. A transformation or analysis between IR levels with an explicit input language, output language, and correctness relation.

Configuration space. A space in which one point represents one complete pose or joint configuration of a movable object or machine.

Conservative approximation. An approximation oriented so that successful checking implies the desired property. Outer sweeps support no-collision claims; inner sweeps support guaranteed-removal claims.

Content addressing. Identifying an artifact by a cryptographic hash of canonical bytes and relevant schema information.

Controller dialect. A particular controller's syntax and operational interpretation, including modal groups, extensions, and lifecycle behavior.

Controller IR. Structured target-controller operations before final textual serialization.

Counterexample. A concrete state, trace, geometry, or input demonstrating that a claim is false.

Cyber-physical system. A system combining discrete software and communication with continuous physical state, timing, sensing, and actuation.

Denotation. The mathematical meaning assigned to a program or IR object.

Denotational semantics. A compositional mapping from syntax to mathematical objects such as sets, functions, relations, or traces.

Dexel. One or more material intervals stored along a sampling ray.

Dimensional type. A type carrying a physical dimension such as length, speed, angle, or spindle rate.

Effect. An interaction beyond pure value computation, including state change, failure, measurement, material removal, I/O, time, or emitted events.

Elaboration. Resolution of source conveniences such as names, defaults, units, frames, and scoped settings into explicit IR.

Enclosure. A set guaranteed to contain an exact but uncertain quantity or geometry.

Error budget. A typed, compositional account of allowed approximation and uncertainty.

Evidence. Data checked to establish a claim, such as interval bounds, abstract states, coverage maps, solver certificates, or trace comparisons.

Feasible set. Every candidate satisfying all hard constraints of an optimization problem.

Final-byte validation. Parsing and interpreting the exact bytes to be deployed and comparing them with the certified controller program.

Frame. A coordinate system with an origin, basis, identity, and transforms to other frames.

Guarantee. A proposition established when its assumptions and preconditions hold.

Hausdorff distance. A set metric measuring the maximum nearest-point discrepancy between two compact sets.

Hoare triple. A contract $\{P\} c \{Q\}$ relating a command's precondition and postcondition.

Inner approximation. A set guaranteed to be contained in the true set.

Intermediate representation. A compiler language used between stages, with syntax, legality, and intended semantics.

Invariant. A proposition true initially and preserved by all relevant transitions.

Kleisli composition. Composition of effectful functions through the sequencing operation of a monad or related effect structure.

Legality predicate. A rule defining which constructs and unresolved features may appear in an IR.

Lipschitz constant. A bound on how much a function can amplify input distance.

Liveness property. A temporal property stating that a desired event eventually occurs under stated assumptions.

Machine IR. A representation in which machine frames, limits, kinematics, and capabilities are concrete but controller syntax is not yet serialized.

Manufacturing intent. A specification of acceptable process outcomes and constraints independent of one concrete path.

Modal state. Controller state that persists across blocks until replaced.

Monad. A mathematical and programming structure for composing computations that carry effects.

Object language. The inert language produced by an authoring phase and consumed by later compiler stages.

Objective function. A quantity optimized among feasible candidates.

Operational semantics. Transition rules describing execution step by step.

Outer approximation. A set guaranteed to contain the true set.

Parse-back validation. Independent parsing of emitted bytes followed by semantic comparison with the source controller IR.

Path. A parameterized geometric curve. It does not by itself specify timing or cutting effect.

Precondition. A proposition that must hold before an operation or theorem application.

Postcondition. A proposition guaranteed after successful execution under the precondition and assumptions.

Proof-carrying CAM. An architecture in which complex producers emit artifacts and evidence checked by a smaller trusted consumer.

Provenance. Structured origin and transformation history for artifacts and sub-artifacts.

Refinement. A relation in which the target introduces detail or removes choices without adding behavior forbidden by the source.

Representation invariant. A property ensuring an in-memory value is internally coherent, narrower than a physical-safety property.

Robust predicate. A geometric predicate whose discrete result remains correct near numerical degeneracy under its input model.

Runtime assurance. Monitoring and intervention by a small trusted component around a more complex or less trusted component.

Safety property. A temporal property stating that a bad event never occurs.

Semantic preservation. Equality, refinement, or bounded correspondence between pass input and output meanings.

Semantic waist. A compact intermediate language separating high-level producers from low-level targets while retaining domain meaning.

Simulation. Execution of one or more modeled traces. Simulation becomes proof only when connected to a theorem covering the required domain.

SSA state token. A single-assignment value that threads dependency through state-changing IR operations.

Staging. Division of program execution into phases, with an earlier phase constructing code or data for a later one.

Stock state. The modeled material remaining at a particular point in the schedule.

Swept volume. The union of all poses occupied by a solid along a trajectory.

Temporal semantics. Meaning defined over traces and properties such as always, eventually, and until.

Trajectory. A time-indexed pose or configuration.

Translation validation. Checking that one actual transformed output correctly relates to one actual input.

Trusted computing base. The components and assumptions that must be correct for the assurance conclusion to hold.

Typestate. Encoding protocol states in types so only state-appropriate operations can be expressed.

Weakest precondition. The least restrictive condition sufficient before a command to guarantee a desired postcondition.

Witness. Producer-supplied data that helps a checker establish an existential or optimization claim.

Selected Exercise Solutions

The solutions are deliberately concise. They show the shape of a correct argument, not the only acceptable formulation.

Chapter 1

Exercise 5. Quarter-circle duration and acceleration. A quarter circle of radius 10 mm has length:

$$L = \frac{\pi R}{2} = 5\pi \approx 15.708 \text{ mm.}$$

At 600 mm/min, duration is:

$$T = L/600 \text{ min} \approx 0.02618 \text{ min} = 1.571 \text{ s.}$$

Speed is 10 mm/s, so centripetal acceleration is:

$$a = v^2/R = 100/10 = 10 \text{ mm/s}^2.$$

Exercise 6. Angular frame uncertainty. With $\delta\theta = 0.0003$ rad at radius 60 mm:

$$\delta x \lesssim r\delta\theta = 60(0.0003) = 0.018 \text{ mm.}$$

This is a bound contribution, not a complete frame error.

Exercise 10. Traverse rule. A simplified big-step rule is:

$$\frac{\text{TraverseReady}(\sigma, \gamma)}{\langle \text{Traverse}(\gamma), \sigma \rangle \Downarrow (\sigma[q := \text{end}(\gamma)], \text{TraverseTrace})}.$$

The unchanged stock component is part of the conclusion.

Exercise 13. Modal ambiguity. Prefix A: G90 G0; prefix B: G91 G1 F100. The same block X10 means an absolute rapid to X=10 after A and an incremental feed of +10 after B.

Exercise 14. Same final stock, different semantics. A cut followed by a probe and a probe followed by the cut can remove the same material, but the probe result differs because the contacted surface changes.

Chapter 2

Exercise 7. Depth design. Modeling Depth as a positive magnitude makes “cut 4 mm deep” natural and avoids double negatives. A separate lowering rule converts it to a frame-relative target coordinate. The cost is that operations requiring signed offsets need another type. Modeling depth as a signed coordinate is more direct for machine motion but easier to misuse across top conventions. The best design usually distinguishes DepthMagnitude from ZCoordinate.

Exercise 15. Path associativity. If exact concatenation is segment-list concatenation and endpoints agree, then both $(r \text{ concat } q) \text{ concat } p$ and $r \text{ concat } (q \text{ concat } p)$ produce the same ordered segment sequence. Identity follows because concatenating the empty segment list changes neither sequence nor endpoints.

Exercise 16. Non-transitive epsilon. For $\varepsilon = 1$, choose points at 0, 0.75, and 1.5. The first two and last two pairs are closer than one; the first and last are not.

Exercise 19. Monad laws. For $\text{pure}(a)$ and bind over deterministic state plus error:

- left identity: $\text{bind}(\text{pure}(a), f) = f(a)$;
- right identity: $\text{bind}(m, \text{pure}) = m$;
- associativity: $\text{bind}(\text{bind}(m, f), g) = \text{bind}(m, a \Rightarrow \text{bind}(f(a), g))$.

Tests should compare result, state, and error behavior.

Exercise 22. Arc linearization contract. Require equal start and end within endpoint bounds, monotone correspondence to source sweep, every chord inside the allowed Hausdorff tube, preserved helical Z interpolation, and a local error sum not exceeding the allocated budget.

Exercise 25. Modal-compression witness. The witness can record the explicit modal state before each source and target block plus a correspondence from compressed target blocks to the source operations they encode. The checker independently interprets both and compares canonical events.

Chapter 3

Exercise 8. Chords for a semicircle. With $R = 25$ mm and $\varepsilon = 0.005$ mm:

$$\theta_{max} = 2 \arccos(1 - 0.005/25) \approx 0.04 \text{ rad} \approx 2.292^\circ.$$

A 180-degree arc needs at least:

$$\lceil \pi/0.04 \rceil = 79$$

chords. A calculator may differ by one because of rounding; use the exact formula before the ceiling.

Exercise 15. Angular contributions. For $\delta\theta = 0.0002$ rad:

- at 20 mm: 0.004 mm;
- at 50 mm: 0.010 mm;
- at 100 mm: 0.020 mm.

Exercise 19. Tool-center rectangle. Inset the 30 by 20 mm pocket by radius 1.5875 mm on every side:

$$[13.5875, 40.4125] \times [15.5875, 32.4125].$$

Exercise 20. Depth layers. Starting at 0, target -3.8, max step 1.3:

-1.3, -2.6, -3.8

The last increment is 1.2 mm.

Exercise 26. Scallop step-over. With $R = 3$ mm and $h = 0.01$ mm:

$$s \approx \sqrt{8Rh} = \sqrt{0.24} \approx 0.490 \text{ mm.}$$

This is a local flat-surface approximation.

Exercise 35. Curvature speed. With $\kappa = 0.2 \text{ mm}^{-1}$ and $a_n^{max} = 500 \text{ mm/s}^2$:

$$v \leq \sqrt{500/0.2} = 50 \text{ mm/s} = 3000 \text{ mm/min.}$$

Other axis and process constraints may be lower.

Chapter 4

Exercise 2. Weakest precondition. Working backward:

- stop spindle requires a valid session and establishes spindle off;
- cut must establish the roughing stock postcondition while ending in a state from which stop is legal;
- start spindle must establish the required spindle state and use an allowed RPM;
- select T1 must begin idle with spindle off and T1 available.

The conjunction also includes homing, known WCS, path safety, feed, travel, and authorization requirements.

Exercise 8. Branching tools. At the merge after branches selecting T1 and T2, the abstract tool value is $\{T1, T2\}$. A T1 cut cannot be proved valid. An explicit select T1 after the merge reduces the state to $\{T1\}$.

Exercise 16. Modal-compression mutations. Delete a required G90, change G1 to G0, omit the first feed, alter arc direction, change one coordinate sign, and remove final M5. Each should cause a specific trace mismatch or final-state failure.

Exercise 25. G54 race. Preflight reads G54 transform T_0 and approves travel. Another client changes G54 to T_1 . The original client starts the job using stale approval. A state epoch tied to WCS changes invalidates the authorization.

Exercise 31. Stopping distance. At 40 mm/s and 400 mm/s²:

$$d_{brake} = v^2/(2a) = 1600/800 = 2 \text{ mm.}$$

At 80 ms latency:

$$d_{latency} = 40(0.08) = 3.2 \text{ mm.}$$

The lower bound is 5.2 mm before additional uncertainty.

Exercise 35. Changed G54. The byte hash identifies controller commands in work coordinates, not their physical placement. Travel and collision claims depend on the work-to-machine transform. A changed transform invalidates those claims even when bytes are unchanged.

Design Checklists

Language and IR checklist

- ☐ JavaScript evaluation is isolated, terminable, deterministic, and capability-limited.
- ☐ The staging result is immutable, serializable, and free of user closures.
- ☐ Every numeric domain validates finite values and appropriate sign/range constraints.
- ☐ Units and frames are explicit before geometric analysis.
- ☐ Every IR level has a documented syntax, semantics, and legality predicate.
- ☐ Paths distinguish geometry from action and timing.
- ☐ Approximate joins produce explicit witnesses and error charges.
- ☐ Machine/process effects are ordered by indexed commands or state tokens.
- ☐ Every pass has an identity, version, relation, configuration hash, and determinism policy.
- ☐ Provenance is total, including synthetic safety motions.

Geometry and planning checklist

- ☐ Tool cutting geometry and full assembly geometry are separate artifacts.
- ☐ Stock, target, protected material, and fixtures are separate semantic inputs.
- ☐ Every spatial representation declares inner/outer/no enclosure meaning.
- ☐ Topological predicates use robust or exact methods appropriate to the input model.
- ☐ Continuous claims are not inferred from sample spacing alone.
- ☐ Error budgets name metrics, frames, subjects, and deterministic/probabilistic interpretation.
- ☐ Links are checked against the stock state that exists when they execute.
- ☐ Scheduling verifies precedence and effect constraints before optimizing cost.
- ☐ Feed planning considers curvature, axes, controller behavior, and process limits.
- ☐ Inconclusive geometry fails closed for policies requiring proof.

Certificate and runtime checklist

- ☐ Every claim names a precise proposition and subject hash.
- ☐ Method, result, bound, assumptions, evidence, and checker identity are separate fields.
- ☐ Evidence types are specific enough to prevent cross-application.
- ☐ The certificate dependency graph is acyclic and fully hash-bound.
- ☐ Policy completeness is checked explicitly.
- ☐ Exact final bytes are independently parsed and interpreted.
- ☐ Preview is generated from or checked against the final deployed artifact.
- ☐ Generic controller commands use a closed grammar and fail-closed effect classification.
- ☐ Command admission and execution are serialized with a state epoch.
- ☐ Upload/start timeouts enter an ambiguous or quarantined state rather than blind retry.
- ☐ Stored bytes are bound to the authorized hash.

- ☐ Stop-class requests remain available and completion means observed cessation or fault.
- ☐ Runtime preflight checks machine, firmware, tool, setup, WCS, homing, alarm, and calibration assumptions.

Further Reading and References

The sources below are selected to form a coherent study path. NIST’s machining reports ground the target domain. The semantics and compiler papers introduce the proof vocabulary. The geometry and optimization papers provide the numerical and planning foundations.

[R1] Thomas R. Kramer, Frederick M. Proctor, and Elena R. Messina. *The NIST RS274/NGC Interpreter, Version 3*. NISTIR 6556, National Institute of Standards and Technology, 2000. <https://www.nist.gov/publications/nist-rs274ngc-interpreter-version-3>.

[R2] Frederick M. Proctor, Thomas R. Kramer, and John L. Michaloski. *Canonical Machining Commands*. NISTIR 5970, National Institute of Standards and Technology, 1997. <https://doi.org/10.6028/NIST.IR.5970>.

[R3] ISO 14649-1. *Industrial Automation Systems and Integration - Physical Device Control - Data Model for Computerized Numerical Controllers - Part 1: Overview and Fundamental Principles*. International Organization for Standardization.

[R4] C. A. R. Hoare. “An Axiomatic Basis for Computer Programming.” *Communications of the ACM* 12, no. 10 (1969): 576-580, 583. <https://doi.org/10.1145/363235.363259>.

[R5] Xavier Leroy. “Formal Verification of a Realistic Compiler.” *Communications of the ACM* 52, no. 7 (2009): 107-115. <https://doi.org/10.1145/1538788.1538814>.

[R6] Patrick Cousot and Radhia Cousot. “Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints.” In *POPL 1977*, 238-252. <https://doi.org/10.1145/512950.512973>.

[R7] Chris Lattner et al. “MLIR: Scaling Compiler Infrastructure for Domain Specific Computation.” In *CGO 2021*, 2-14. <https://doi.org/10.1109/CGO51591.2021.9370308>.

[R8] Leslie Lamport. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16, no. 3 (1994): 872-923. <https://doi.org/10.1145/177492.177726>.

[R9] Walid Taha and Tim Sheard. “MetaML and Multi-Stage Programming with Explicit Annotations.” *Theoretical Computer Science* 248, nos. 1-2 (2000): 211-242. [https://doi.org/10.1016/S0304-3975\(00\)00053-0](https://doi.org/10.1016/S0304-3975(00)00053-0).

[R10] Eugenio Moggi. “Notions of Computation and Monads.” *Information and Computation* 93, no. 1 (1991): 55-92. [https://doi.org/10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).

[R11] Robert E. Strom and Shaula Yemini. “Typestate: A Programming Language Concept for Enhancing Software Reliability.” *IEEE Transactions on Software Engineering* SE-12, no. 1 (1986): 157-171. <https://doi.org/10.1109/TSE.1986.6312929>.

[R12] Robert Atkey. “Parameterised Notions of Computation.” *Journal of Functional Programming* 19, nos. 3-4 (2009): 335-376. <https://doi.org/10.1017/S095679680900728X>.

[R13] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. “Efficiently Computing Static Single Assignment Form and the Control Dependence Graph.” *ACM Transactions on Programming Languages and Systems* 13, no. 4 (1991): 451-490. <https://doi.org/10.1145/115372.115320>.

- [R14]** Amir Pnueli, Michael Siegel, and Eli Singerman. “Translation Validation.” In *TACAS 1998*, 151-166. <https://doi.org/10.1007/BFb0054170>.
- [R15]** George C. Necula. “Proof-Carrying Code.” In *POPL 1997*, 106-119. <https://doi.org/10.1145/263699.263712>.
- [R16]** Andrew W. Appel. “Foundational Proof-Carrying Code.” In *LICS 2001*, 247-256. <https://doi.org/10.1109/LICS.2001.932501>.
- [R17]** Jonathan Richard Shewchuk. “Adaptive Precision Floating-Point Arithmetic and Fast Robust Geometric Predicates.” *Discrete & Computational Geometry* 18 (1997): 305-363. <https://doi.org/10.1007/PL00009321>.
- [R18]** Tomás Lozano-Pérez. “Spatial Planning: A Configuration Space Approach.” *IEEE Transactions on Computers* C-32, no. 2 (1983): 108-120. <https://doi.org/10.1109/TC.1983.1676196>.
- [R19]** Ramon E. Moore, R. Baker Kearfott, and Michael J. Cloud. *Introduction to Interval Analysis*. SIAM, 2009. <https://doi.org/10.1137/1.9780898717716>.
- [R20]** Masatomo Inui, Takashi Sakurai, and Nobuyuki Umezu. “Data Conversion Technology between Triple Dixel Model and Polygonal Model.” *Journal of the Japan Society for Precision Engineering* 76, no. 2 (2010): 226-231. <https://doi.org/10.2493/jjspe.76.226>.
- [R21]** Weihang Zhang and Ming-Chuan Leu. “Surface Reconstruction Using Dixel Data from Three Sets of Orthogonal Rays.” *Journal of Computing and Information Science in Engineering* 9, no. 1 (2009): 011008. <https://doi.org/10.1115/1.3086034>.
- [R22]** J. A. Sethian. “A Fast Marching Level Set Method for Monotonically Advancing Fronts.” *Proceedings of the National Academy of Sciences* 93, no. 4 (1996): 1591-1595. <https://doi.org/10.1073/pnas.93.4.1591>.
- [R23]** Ron Kimmel and J. A. Sethian. “Computing Geodesic Paths on Manifolds.” *Proceedings of the National Academy of Sciences* 95, no. 15 (1998): 8431-8435. <https://doi.org/10.1073/pnas.95.15.8431>.
- [R24]** Ilker Kucukoglu, Tulin Gunduz, Fatma Balkancioglu, Emine Chousein Topal, and Oznur Sayim. “Application of Precedence Constrained Travelling Salesman Problem Model for Tool Path Optimization in CNC Milling Machines.” *An International Journal of Optimization and Control: Theories & Applications* 9, no. 3 (2019): 59-68. <https://doi.org/10.11121/ijocta.01.2019.00662>.
- [R25]** Hung Pham and Quang-Cuong Pham. “A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis.” *IEEE Transactions on Robotics* 34, no. 3 (2018): 645-659. <https://doi.org/10.1109/TRO.2018.2819195>.
- [R26]** Rajeev Alur, Costas Courcoubetis, Thomas A. Henzinger, and Pei-Hsin Ho. “The Algorithmic Analysis of Hybrid Systems.” *Theoretical Computer Science* 138, no. 1 (1995): 3-34. [https://doi.org/10.1016/0304-3975\(94\)00202-T](https://doi.org/10.1016/0304-3975(94)00202-T).
- [R27]** J. Tanner Slagel, Lauren M. White, Aaron Dutle, César A. Muñoz, and Nicolas Crespo. “A Formal Verification Framework for Runtime Assurance.” In *NASA Formal Methods 2024*, 322-328. https://doi.org/10.1007/978-3-031-60698-4_19.
- [R28]** Thomas R. Kramer and Frederick M. Proctor. *Feature-Based Control of a Machining Center*. NISTIR 5926, National Institute of Standards and Technology, 1996. <https://www.nist.gov/publications/feature-based-control-machining-center>.

Source note

The implementation examples and architecture discussion were informed by the supplied dropcut-studio.zip snapshot, including its semantic CAM design notes and the packages for IR, planning, geometry, analysis, postprocessing, scripting, and the Makera Z1 controller client. The textbook deliberately abstracts

from one code revision. Source-specific claims should be rechecked against the revision being built or operated.